PONTIFÍCIA UNIVERSIDADE CATÓLICA DO PARANÁ
ESCOLA POLITÉCNICA — BACHARELADO EM ENGENHARIA DE SOFTWARE
Disciplina: Arquitetura de Soluções Cloud — 6º Semestre / 2026

ORQUESTRACOMPRAS

Plataforma Inteligente de Monitoramento de Mercado e Otimização Orçamentária

Documento de Arquitetura de Software (DAS) — Padrão arc42
Status: Documento Final de Arquitetura | **Padrão:** arc42

Integrantes do Grupo de Engenharia:
Matheus Lucas da Silva — Matrícula: 40095149
Lucas Schapiewski Moro — Matrícula: 40090687
Angely Alejandra Zambrano Munoz — Matrícula: 40094261
Erick Werner Stalhlschimidt

Curitiba — PR
Outubro de 2026

SUMÁRIO

O documento está organizado nas 11 seções principais do modelo arc42, complementadas por um glossário de termos e siglas.

- **1. Introdução e Objetivos**
  - 1.1 Contextualização da Plataforma e Problema de Negócio
  - 1.2 Objetivos de Negócios Subjacentes
  - 1.3 Software Architecture Canvas
  - 1.4 Visão Geral dos Requisitos
    - 1.4.1 Introdução e Escopo Funcional
    - 1.4.2 Motivação sob a Perspectiva das Partes
    - 1.4.3 Matriz Consolidada de Requisitos Funcionais (RF)
  - 1.5 Objetivos de Qualidade da Arquitetura (ISO/IEC 25010)
  - 1.6 Partes Interessadas (Stakeholders) e Expectativas
- **2. Restrições Arquiteturais**
  - 2.1 Restrições Técnicas
  - 2.2 Restrições Organizacionais
  - 2.3 Convenções, Políticas e Diretrizes de Engenharia
- **3. Contexto e Escopo**
  - 3.1 Contexto Negocial
  - 3.2 Contexto Técnico
- **4. Estratégia de Solução**
  - 4.1 Decisões Tecnológicas Fundamentais
  - 4.2 Decomposição de Alto Nível e Padrões Arquiteturais
  - 4.3 Estratégias para Cumprimento das Metas de Qualidade
  - 4.4 Decisões Organizacionais e Processos de Engenharia
  - 4.5 Matriz de Alinhamento Estratégico
- **5. Visão de Blocos de Construção**
  - 5.1 Visão Sistêmica Geral de Caixa Branca (Referência ao C4 Nível 1)
  - 5.2 Nível 2: Decomposição em Contêineres (C4 Nível 2)
  - 5.3 Nível 3: Decomposição do Monitoring Service (C4 Nível 3)
- **6. Visão de Tempo de Execução**

    - 6.1 Cenário 1: Ciclo Autônomo de Varredura e Avaliação por IA (Agente com Fallback)
    - 6.2 Cenário 2: Cadastro, Autenticação e Emissão de Token JWT
    - 6.3 Cenário 3: Criação de Orçamento e Sincronização por Eventos
    - 6.4 Cenário 4: Encerramento de Conta e Tratamento de Privacidade (LGPD)
    - 6.5 Aspectos Relevantes da Execução de Runtime
- **7. Visão de Implantação**
    - 7.1 Nível de Infraestrutura 1: Topologia Cloud e Ambientes
    - 7.2 Mapeamento de Artefatos de Software para Nós de Infraestrutura
- **8. Conceitos Transversais**
    - 8.1 Modelo de Domínio e Persistência de Dados (DER e Dicionário NoSQL)
        - 8.1.1 Dicionário de Dados das Coleções e Documentos BSON
        - 8.1.2 Validação de Esquema, Índices e Atomicidade
    - 8.2 Padrões de Projeto e Estrutura Estática (Diagrama de Classes UML)
    - 8.3 Modularização Física e Camadas de Código (Diagrama de Componentes UML)
    - 8.4 Diretrizes de Interface e Experiência do Usuário (UX/UI)
    - 8.5 Conceitos de Segurança, Criptografia e Proteção da Informação
    - 8.6 Resiliência Cognitiva: Circuit Breaker e Fallback Heurístico Local
    - 8.7 Tratamento Padronizado de Erros e Validações HTTP
    - 8.8 Conceitos de Observabilidade e Telemetria
    - 8.9 Conceitos de Testabilidade e Qualidade de Código
    - 8.10 Conceitos de Build, Versionamento e Entrega Contínua (CI/CD)
    - 8.11 Resumo dos Conceitos Transversais
- **9. Decisões Arquiteturais (ADRs)**
- **10. Requisitos de Qualidade**
    - 10.1 Árvore de Qualidade (ISO/IEC 25010)
    - 10.2 Cenários de Qualidade Mensuráveis
    - 10.3 Análise de Trade-Offs Arquiteturais (Método ATAM)
    - 10.4 Análise dos Resultados ATAM
    - 10.5 Matriz de Priorização dos Atributos de Qualidade
- **11. Riscos e Débitos Técnicos**
    - 11.1 Riscos Arquiteturais e Mitigações
    - 11.2 Débitos Técnicos Identificados e Planos de Ação
    - 11.3 Matriz de Riscos Relacionados à Segurança da Informação
- **12. Glossário**

LISTA DE FIGURAS

- **Figura 01:** Software Architecture Canvas — visão executiva do OrquestraCompras (Seção 1.3)
- **Figura 02:** C4 Nível 1 — Contexto do sistema e parceiros externos (Seção 3.1)
- **Figura 03:** C4 Nível 2 — Contêineres e limites de responsabilidade da solução (Seção 5.2)
- **Figura 04:** C4 Nível 3 — Componentes internos do Monitoring Service (Seção 5.3)
- **Figura 05:** UML Sequência — Monitoramento, IA, mock e fallback heurístico (Seção 6.1)
- **Figura 06:** UML Sequência — Cadastro, autenticação JWT e bloqueio de acesso (Seção 6.2)
- **Figura 07:** UML Sequência — Criação de orçamento e sincronização por eventos (Seção 6.3)
- **Figura 08:** UML Sequência — Encerramento de conta e tratamento assíncrono de privacidade (Seção 6.4)
- **Figura 09:** UML de Implantação Cloud — unidades independentes e infraestrutura gerenciada (Seção 7.1)
- **Figura 10:** DER lógico NoSQL — Database per Service com snapshots e marcadores de privacidade (Seção 8.1)
- **Figura 11A:** UML de Classes — Identity Service, Budget Service e contratos JSON versionados (Seção 8.2)
- **Figura 11B:** UML de Classes — Domínio do Monitoring Service (Seção 8.2)
- **Figura 12:** UML de Componentes — Clean Architecture, Vertical Slice e adaptadores (Seção 8.3)
- **Figura 13:** Fluxo de Circuit Breaker e Fallback Heurístico — decisão controlada pelo domínio (Seção 8.6)
- **Figura 14:** Árvore de Qualidade ISO/IEC 25010 — atributos prioritários (Seção 10.1)

LISTA DE TABELAS

- **Tabela 1:** Matriz consolidada de requisitos funcionais RF01 a RF14 (Seção 1.4)
- **Tabela 2:** Objetivos de qualidade prioritários da arquitetura (Seção 1.5)
- **Tabela 3:** Partes interessadas e expectativas (Seção 1.6)
- **Tabela 4:** Restrições técnicas de arquitetura (Seção 2.1)
- **Tabela 5:** Restrições organizacionais e operacionais (Seção 2.2)
- **Tabela 6:** Convenções, políticas e diretrizes de engenharia (Seção 2.3)
- **Tabela 7:** Entradas e saídas do contexto negocial (Seção 3.1)
- **Tabela 8:** Protocolos, formatos e regras entre fronteiras (Seção 3.2)
- **Tabela 9:** Decomposição de alto nível da solução (Seção 4.2)
- **Tabela 10:** Estratégias e mecanismos de qualidade (Seção 4.3)
- **Tabela 11:** Decisões, atributos de qualidade e restrições (Seção 4.5)
- **Tabela 12:** Especificação dos contêineres e infraestrutura lógica (Seção 5.2)
- **Tabela 13:** Componentes do Monitoring Service e dependências (Seção 5.3)
- **Tabela 14:** Contratos HTTP e eventos da arquitetura (Seção 5.3)
- **Tabela 15:** Matriz de responsabilidades — cenário de monitoramento (Seção 6.1)
- **Tabela 16:** Matriz de responsabilidades — cadastro e JWT (Seção 6.2)
- **Tabela 17:** Matriz de responsabilidades — criação e EDA (Seção 6.3)
- **Tabela 18:** Matriz de responsabilidades — encerramento distribuído (Seção 6.4)
- **Tabela 19:** Artefatos, nós de implantação e comunicação (Seção 7.2)
- **Tabela 20:** Dicionário NoSQL — db_identity.usuarios (Seção 8.1)
- **Tabela 21:** Dicionário NoSQL — db_budgets.orcamentos (Seção 8.1)
- **Tabela 22:** Dicionário NoSQL — subdocumentos e auditoria Budget (Seção 8.1)
- **Tabela 23:** Dicionário NoSQL — db_monitoring.assinaturasMonitoramento (Seção 8.1)
- **Tabela 24:** Dicionário NoSQL — tentativas, oportunidades e privacidade Monitoring (Seção 8.1)
- **Tabela 25:** Elementos do domínio, objetos de valor e serviços (Seção 8.2)
- **Tabela 26:** Diretrizes UX/UI por fluxo (Seção 8.4)
- **Tabela 27:** Mecanismos de segurança e riscos tratados (Seção 8.5)
- **Tabela 28:** Respostas HTTP e decisões de negócio (Seção 8.7)
- **Tabela 29:** Plano de teste por nível e evidência (Seção 8.9)
- **Tabela 30:** Resumo dos conceitos transversais (Seção 8.11)
- **Tabela 31:** Cenários de qualidade mensuráveis SC-01 a SC-14 (Seção 10.2)

- **Tabela 32:** Avaliação arquitetural ATAM-01 (Seção 10.3)
- **Tabela 33:** Avaliação arquitetural ATAM-02 (Seção 10.3)
- **Tabela 34:** Avaliação arquitetural ATAM-03 (Seção 10.3)
- **Tabela 35:** Avaliação arquitetural ATAM-04 (Seção 10.3)
- **Tabela 36:** Avaliação arquitetural ATAM-05 (Seção 10.3)
- **Tabela 37:** Prioridades dos atributos de qualidade (Seção 10.5)
- **Tabela 38:** Riscos arquiteturais e planos de mitigação (Seção 11.1)
- **Tabela 39:** Débitos técnicos e ações corretivas prioritárias (Seção 11.2)
- **Tabela 40:** Ameaças de segurança e mecanismos de defesa (Seção 11.3)
- **Tabela 41:** Glossário de termos e siglas utilizados (Seção 12)

1. Introdução e Objetivos

O **OrquestraCompras** é uma aplicação web de apoio à decisão de compras de tecnologia. O comprador registra um projeto de aquisição, um teto financeiro, componentes desejados, alternativas aceitáveis e lojas de confiança. A plataforma acompanha catálogos e disponibilidade e apresenta oportunidades cuja combinação seja **completa, compatível e financeiramente viável**. Quando nenhuma combinação atende aos requisitos, recomenda aguardar e registra a tentativa, sem apresentar uma cesta incompleta como compra possível.

A arquitetura do OrquestraCompras é formada por um **Web Shell** que compõe os microfrontends **MFE Conta** e **MFE Orçamentos**, um **Gateway/BFF** e três microsserviços serverless — **Identity**, **Budget** e **Monitoring** — executados em Azure Functions. A persistência utiliza três bancos lógicos exclusivos no MongoDB Atlas. A integração assíncrona utiliza Azure Storage Queues, eventos JSON versionados, *transactional outbox* e consumidores idempotentes. O motor de recomendação utiliza Gemini API quando configurado, mock local para operação controlada e fallback heurístico determinístico como contingência.

Abrangência arquitetural. Este documento descreve as responsabilidades, os contratos, os mecanismos operacionais e as decisões arquiteturais da plataforma de forma integrada. As seções 8, 10 e 11 registram políticas técnicas, cenários de qualidade e riscos associados ao funcionamento do sistema. Metas de desempenho são apresentadas como critérios de avaliação, e não como estatísticas observadas.

1.1 Contextualização da Plataforma e Problema de Negócio

A cotação manual entre vários varejistas fragmenta a visão do custo global, especialmente quando o comprador aceita peças tecnicamente equivalentes. As lojas podem oferecer preços distintos, estoques voláteis e condições de entrega diferentes. Uma oferta isolada não significa que seja possível adquirir **todos os componentes obrigatórios** dentro do teto. A aplicação centraliza essa análise e apresenta links e justificativas da oportunidade encontrada. A decisão final e a aquisição permanecem com o usuário.

1.2 Objetivos de Negócios Subjacentes

- **Eficiência financeira:** identificar cestas completas e viáveis que respeitem o limite informado e evidenciem economia estimada.
- **Monitoramento automático:** avaliar periodicamente os projetos ativos sem exigir navegação contínua pelo comprador.
- **Flexibilidade:** permitir alternativas de hardware, itens obrigatórios e seleção de lojas confiáveis.
- **Resiliência:** manter a análise útil quando a LLM estiver indisponível, sem inventar preços ou peças e sem afirmar ótimo global não demonstrado.
- **Privacidade e autonomia de evolução:** isolar contas e dados, coordenar o encerramento distribuído e permitir a atualização independente dos módulos sem alterar as regras centrais do negócio.
- **Sustentabilidade econômica:** priorizar serviços com cotas gratuitas, com monitoramento e limites de consumo; **R$ 0 não é garantia de cobrança futura zero**.

1.3 Software Architecture Canvas

O Canvas sintetiza o *business case*, o contexto de negócio, os requisitos de qualidade, as decisões estruturais, as restrições e os riscos da solução.

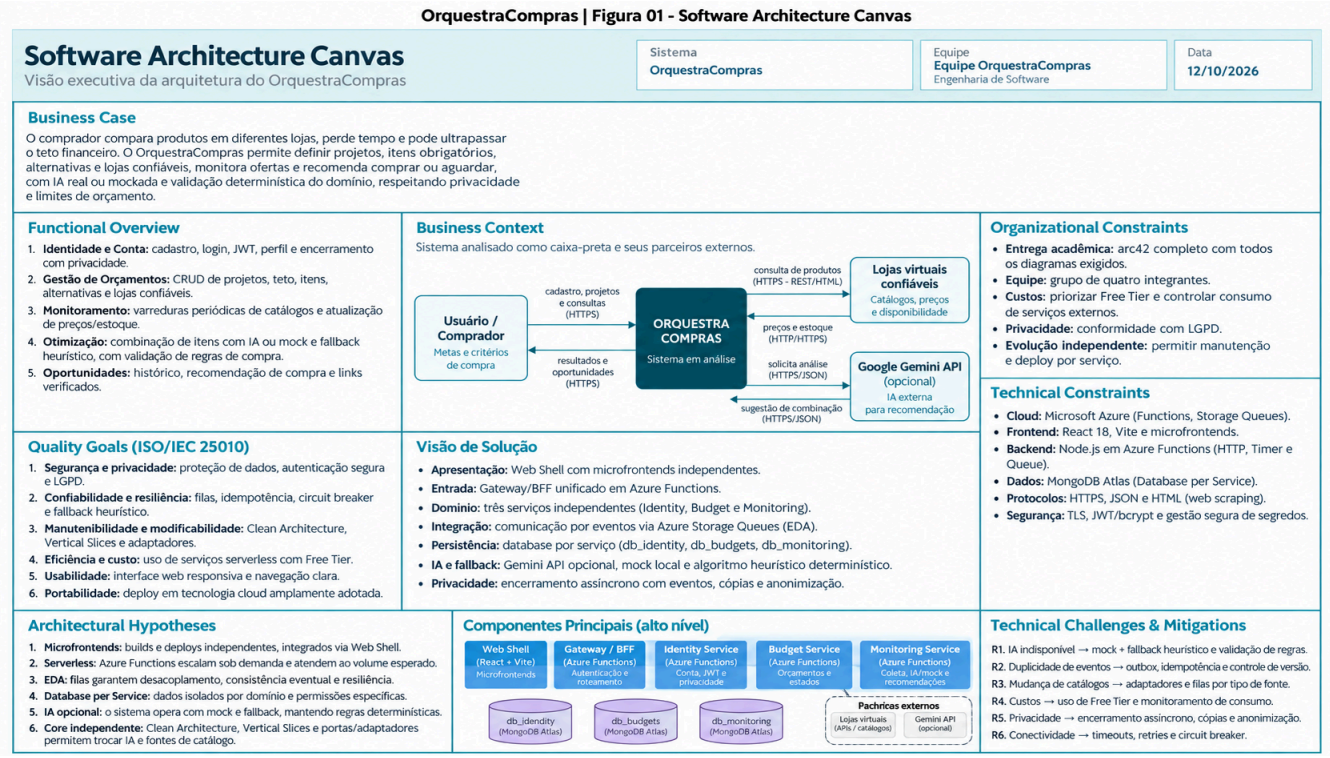


Figura 01: Software Architecture Canvas — visão executiva do OrquestraCompras.

1.4 Visão Geral dos Requisitos

1.4.1 Introdução e Escopo Funcional

O sistema organiza suas funcionalidades em três eixos integrados: **identidade e acesso**, **gestão de orçamentos** e **monitoramento com otimização e oportunidades**. O Web Shell reúne as interfaces de conta e orçamentos, enquanto os serviços de domínio executam cadastro, autenticação, gestão dos projetos, sincronização de snapshots, análise de ofertas e tratamento de privacidade. Os fluxos síncronos e assíncronos são detalhados nas visões de execução.

1.4.2 Motivação sob a Perspectiva das Partes

- **Comprador:** controlar suas preferências, receber resultados verificáveis e impedir acesso de terceiros a seus dados.
- **Equipe de desenvolvimento:** evoluir conta, orçamento e motor de pesquisa de forma independente, com interfaces estáveis.
- **Arquitetos e revisores técnicos:** compreender a decomposição de software, a rastreabilidade das decisões e a consistência entre interfaces e diagramas.
- **Operação e privacidade:** observar falhas, rastrear eventos, respeitar o bloqueio de uma conta e tratar solicitações de titulares sem depender de transação global entre bancos.

1.4.3 Matriz Consolidada de Requisitos Funcionais (RF)

Tabela 1: Matriz consolidada de requisitos funcionais RF01 a RF14.

Código	Área	Requisito	Descrição
`RF01`	Identidade	Cadastro de contas	Autorregistro com email único, validação e senha transformada por bcrypt.
`RF02`	Identidade	Autenticação JWT	Login, emissão de JWT com expiração e autenticação nos recursos protegidos.
`RF03`	Identidade	Gestão de credenciais	Alteração/redefinição de senha, proteção de sessão e revogação quando aplicável.
`RF04`	Privacidade	Ciclo de vida da conta	Atualização de perfil, exportação quando aplicável e solicitação de encerramento com bloqueio de acesso e processamento distribuído.
`RF05`	Orçamentos	CRUD de projetos	Criar, listar, alterar, pausar, concluir e excluir projetos de compra com teto financeiro.
`RF06`	Orçamentos	Itens e equivalências	Definir categorias, requisitos, produtos alternativos aceitos e obrigatoriedade.
`RF07`	Orçamentos	Lojas autorizadas	Manter lojas e limitar a coleta à seleção de cada projeto.
`RF08`	Orçamentos	Estados do orçamento	Controlar MONITORANDO, PAUSADO, ALERTA_DISPARADO, CONCLUIDO e EXPIRADO.
`RF09`	Monitoramento	Histórico de tentativas	Registrar tempo, origem de cálculo, resultado, motivo e versão do orçamento.
`RF10`	Auditoria	Trilha de alterações	Registrar mudanças relevantes, autor, antes/depois conforme política de retenção.

Código	Área	Requisito	Descrição
`RF11`	Monitoramento	Coleta periódica	Consultar preços e disponibilidade em fontes autorizadas ou catálogos de teste.
`RF12`	Otimização	Recomendação com IA/mock	Gerar uma combinação candidata com catálogo e restrições por adaptação externa ou mock local.
`RF13`	Resiliência	Fallback heurístico	Contornar falhas ou ausência da IA externa por regra local determinística; validar a combinação candidata pelas regras de domínio.
`RF14`	Oportunidades	Painel de resultados	Exibir histórico, preço total, economia estimada, situação de compra e links dos produtos.

Regras funcionais invariantes: (i) uma oportunidade de compra exige todos os itens obrigatórios disponíveis e compatíveis; (ii) `valorTotalCalculado <= valorLimite`; (iii) resultados referenciam a versão do orçamento em que foram gerados; (iv) alterações e encerramento bloqueiam a aplicação indevida de eventos obsoletos; (v) recomendações da LLM e do mock nunca dispensam validação pelo domínio; (vi) custo total depende das informações efetivamente coletadas, não se presumindo frete/estoque que a fonte não forneça.

1.5 Objetivos de Qualidade da Arquitetura (ISO/IEC 25010)

Tabela 2: Objetivos de qualidade prioritários da arquitetura.

Prioridade	Atributo	Objetivo verificável
1	Segurança e privacidade	Credenciais com bcrypt, JWT assinado e verificado, autorização por titular; revogação/estado checados; bancos isolados por privilégios.
2	Confiabilidade e resiliência	Outbox e consumidores idempotentes; registro de tentativas e falhas; IA externa substituível por mock/fallback; ausência de recomendações falsas.
3	Manutenibilidade e modificabilidade	Domínio independente de Azure, LLM e MongoDB; vertical slices; MFE e Function Apps com implantação independente.
4	Eficiência e custo	UI distribuída estaticamente, leituras de snapshots, varreduras assíncronas e acompanhamento de cotas de serviços externos.

1.6 Partes Interessadas (Stakeholders) e Expectativas

Tabela 3: Partes interessadas e expectativas.

Parte interessada	Canal	Expectativa
Comprador	MFE Conta e MFE Orçamentos	Segurança, configurações claras, resultados consistentes, encerramento auditável.
Equipe de engenharia (4 estudantes)	GitHub, VS Code, Azure, arc42	Código compatível com os diagramas, contratos versionados e execução reprodutível.
Arquitetos e auditores técnicos	arc42 e repositório	Rastreabilidade de decisões, C4 níveis 1 a 3, UML de sequência/classes/componentes, DER e Canvas legíveis.
Administração de nuvem / operação	Azure Portal e MongoDB Atlas	Custos controlados, falhas observáveis, segredos protegidos, filas monitoradas.
Titular de dados	Fluxo de conta e privacidade	Bloqueio de uso e tratamento adequado de dados pessoais segundo política e legislação.

2. Restrições Arquiteturais

As restrições estabelecem as condições técnicas, organizacionais e de engenharia observadas pelos componentes do OrquestraCompras. Estão agrupadas em restrições técnicas (RT), organizacionais (RO) e convenções de engenharia (RC).

2.1 Restrições Técnicas

Tabela 4: Restrições técnicas de arquitetura.

ID	Restrição	Detalhamento e impacto
`RT-01`	Serverless	Azure Functions, Node.js LTS; Function Apps independentes para Gateway, Identity, Budget e Monitoring.
`RT-02`	Microfrontends	Web Shell React/Vite com MFE Conta e MFE Orçamentos, builds e publicações autônomas.
`RT-03`	Gateway/BFF	Entrada pública lógica única; autenticação, composição e roteamento HTTP/JSON; não pressupõe serviço pago API Management.
`RT-04`	Database per Service	MongoDB Atlas com db_identity, db_budgets, db_monitoring e usuários técnicos sem acesso cruzado.
`RT-05`	Documentos embutidos	Itens e lojas no orçamento; itens de oportunidade no agregado correspondente; mudanças atômicas por documento.
`RT-06`	EDA	Azure Storage Queues, eventos com versão, eventId, at-least-once e idempotência de consumidor.
`RT-07`	Outbox local	Persistência do agregado e do evento pendente na mesma escrita de documento, dispatcher por serviço e reenvio controlado.
`RT-08`	Transportes seguros	HTTPS/TLS para tráfego público e interno; TLS para MongoDB; credenciais por ambiente.
`RT-09`	JWT e conta	JWT com assinatura, validade e claims mínimos; BFF verifica status/revogação no Identity para ações protegidas.
`RT-10`	Privacidade	Solicitação de encerramento aceita assincronamente; bloqueio, fan-out explícito e confirmações posteriores.
`RT-11`	IA substituível	Gemini externo opcional; mock local e fallback heurístico; saída validada pelo domínio.
`RT-12`	Coleta respeitosa	Catálogos por APIs quando disponíveis; HTML apenas quando permitido; limites, timeouts e tratamento de falhas.
`RT-13`	Runtime stateless	Functions sem dependência de disco local persistente nem estado em memória como fonte de verdade.
`RT-14`	Conexão MongoDB	Reutilização de MongoClient por instância e limites de pool/concurrency; não compartilhar credenciais entre serviços.
`RT-15`	Observabilidade	eventId, correlationId, idOrcamento e métricas sem registros pessoais desnecessários.
`RT-16`	API e contratos	Rotas públicas sob /api/v1, schemas JSON e gestão explícita de compatibilidade de contratos.

ID	Restrição	Detalhamento e impacto
`RT-17`	Proteção contra atrasos	Controle de versão de orçamento e marcadores de privacidade impedem reativação de dados por eventos antigos.

2.2 Restrições Organizacionais

Tabela 5: Restrições organizacionais e operacionais.

ID	Restrição	Justificativa
`RO-01`	Priorizar free tier	Arquitetura busca custo reduzido; tarifas, limites e habilitação dos recursos precisam ser verificados na conta real.
`RO-02`	Documentação arc42	Organização em 11 seções arquiteturais, glossário complementar e diagramas localizados em seus contextos.
`RO-03`	Modelagem arquitetural	C4 níveis 1, 2 e 3; UML de sequência, classes e componentes no nível 4; DER e Software Architecture Canvas.
`RO-04`	Responsabilidades técnicas	Equipe de quatro integrantes, com documentação e responsabilidades de manutenção compartilhadas.
`RO-05`	Independência de publicação	Cada microfrontend e cada Function App possui ciclo autônomo de build, teste, implantação e rollback.
`RO-06`	Ambientes reprodutíveis	Scripts npm, Azure Functions Core Tools, configuração local não versionada e documentação de bootstrap.
`RO-07`	Prioridade funcional	Manter cadastro de compra, validação de limite e cenário real/mock como núcleo do produto.
`RO-08`	Documentação e comunicação	Documento arquitetural consolidado e diagramas legíveis, com rastreabilidade entre seções.

2.3 Convenções, Políticas e Diretrizes de Engenharia

Tabela 6: Convenções, políticas e diretrizes de engenharia.

ID	Diretriz	Norma de aplicação
`RC-01`	Clean Architecture	Domínio puro sem imports de Azure, banco ou HTTP; aplicação depende de portas, infraestrutura as implementa.
`RC-02`	Vertical Slice	Cada caso de uso contém entrada, aplicação, DTO/validação e teste próprios sem criar dependência circular.
`RC-03`	Privacidade	Mínimo de dados pessoais; segregação de contextos; retenção por finalidade e base legal, sem prometer apagamento imediato irrestrito.
`RC-04`	Credenciais	bcrypt com custo configurável a partir de 10, armazenamento de hash e segredo fora do código.
`RC-05`	Versionamento	API /api/v1, DTOs EventoOrcamentoV1, EventoOportunidadeV1, EventoContaEncerramentoSolicitadoV1 e PrivacidadeProcessadaV1.
`RC-06`	Erros padronizados	Envelope de erro com timestamp, code, message e correlationId; códigos HTTP consistentes.
`RC-07`	Auditoria sem PII	Logs estruturados, campos mínimos e mascaramento de email, token e credenciais.
`RC-08`	Autorização	Todas as leituras e escritas de dados de usuário recebem o titular do token validado; comparar com usuário do recurso.
`RC-09`	Testabilidade	Mocks para lojas/IA/filas/banco, testes unitários de regras, contrato e integração independentes.
`RC-10`	Nomenclatura	JavaScript camelCase para métodos/vars; PascalCase para classes; entidades/contratos conforme diagramas.
`RC-11`	Publicação idempotente	eventId e operacaoId únicos; reenvio é permitido, efeitos duplicados não.
`RC-12`	Política de segredos	local.settings.json ignorado; Azure Application Settings; rotação e privilégio mínimo.
`RC-13`	Frontend seguro	CORS restritivo é defesa auxiliar, não autorização; CSP, escaping, HTTPS e token em memória como decisão inicial.
`RC-14`	Política de estados	StatusConta e StatusOrcamento como enums; transições validadas pelo respectivo serviço proprietário.

3. Contexto e Escopo

O OrquestraCompras é responsável por transformar metas de compra em resultados consultáveis. A fronteira externa contém o navegador do usuário e dois parceiros: plataformas de comércio eletrônico e, opcionalmente, um provedor externo de inferência. O **mock é interno** e não aparece como sistema externo no C4 Nível 1.

3.1 Contexto Negocial

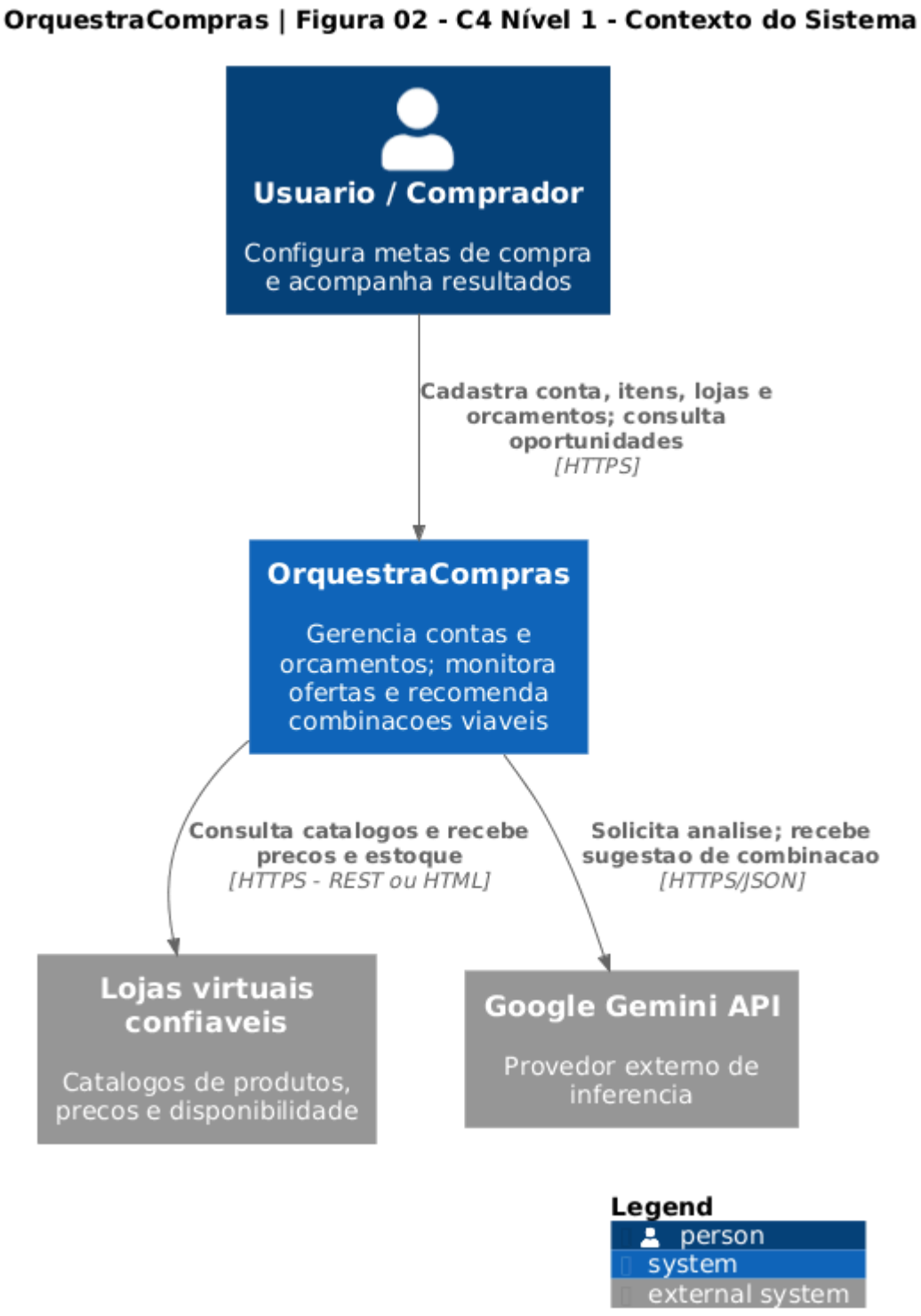


Figura 02: C4 Nível 1 — Contexto do sistema e parceiros externos.

Tabela 7: Entradas e saídas do contexto negocial.

Parceiro	Informações enviadas ao OrquestraCompras	Informações recebidas do OrquestraCompras
Usuário/Comprador	Dados de perfil, credenciais, teto, itens, alternativas, lojas, comandos de conta.	Resultado da autenticação, orçamentos, tentativas, oportunidades e estado de encerramento.
Lojas confiáveis	Catálogo, estoque, preços e URLs disponíveis na origem.	Consultas de produtos conforme as lojas permitidas e políticas de acesso.
Google Gemini API (opcional)	Sugestão de combinação e justificativa estruturada; pode falhar/ser substituído.	Prompt com catálogo coletado e restrições de compra, sem credenciais do usuário.

3.2 Contexto Técnico

Tabela 8: Protocolos, formatos e regras entre fronteiras.

Fluxo	Canal e formato	Segurança / consistência
Navegador → Web Shell/MFEs	HTTPS / HTML, JS, CSS; módulos remotos	CSP/HTTPS, contratos de carregamento e versões compatíveis.
MFEs → Gateway/BFF	HTTPS / JSON / Bearer JWT	CORS de origem autorizada, assinatura e expiração verificadas.
BFF → serviços proprietários	HTTPS / JSON (APIs privadas autenticadas)	Identidade entre serviços, autorização por titular e sem acesso direto aos bancos.
Identity/Budget/Monitoring → bancos próprios	MongoDB TLS / BSON	Usuários db-specific, pool por Function App e índices adequados.
Publicadores/consumidores → Storage Queues	Azure Queue protocol / mensagens JSON versionadas	At-least-once, visibilidade, repetição, poison queue e idempotência.
Monitoring → lojas	HTTPS / REST ou HTML	Limite de taxa, coleta autorizada, tratamento de indisponibilidade.
Monitoring → Gemini (opcional)	HTTPS / JSON	Chave secreta em ambiente, timeout e Circuit Breaker; mock local não envia HTTP.

Limite de confiança: a interface cliente nunca acessa diretamente bancos MongoDB, filas ou a chave da LLM. A autorização é feita no serviço responsável pelo dado, além das validações de borda. Os três bancos lógicos podem residir no mesmo cluster Atlas, mas não constituem um único banco compartilhado.

4. Estratégia de Solução

A estratégia combina **decomposição por domínio**, autonomia de implantação e contratos verificáveis. O sistema distribui quatro unidades de backend em Azure Function Apps independentes e três publicações estáticas de frontend, evitando servidores dedicados permanentemente ativos. Essa organização isola ciclos de manutenção e permite dimensionamento adequado aos diferentes perfis de carga.

4.1 Decisões Tecnológicas Fundamentais

Microfrontends com Web Shell

O navegador carrega o **Web Shell**, responsável pelo roteamento, layout e composição dos módulos **MFE Conta** e **MFE Orçamentos**. Cada MFE possui build e publicação independentes; um manifesto de carregamento de módulos remotos controla URLs, versões, cache e compatibilidade. O host controla navegação e elementos compartilhados; os MFEs não devem importar implementações internas uns dos outros. A separação mantém responsabilidades de apresentação alinhadas aos domínios de conta e orçamento, com contratos explícitos entre módulos.

Gateway e Backend for Frontend (BFF)

O **Gateway/BFF** é a entrada pública lógica: recebe HTTPS, extrai e verifica JWT, valida payloads e compõe respostas direcionando comandos aos serviços proprietários. Não é, por si só, uma instância do produto Azure API Management. Uma chamada de serviço protegida deve autenticar o chamador, não confiar apenas no cabeçalho `usuarioId` declarado pelo cliente.

Três microsserviços em FaaS

- **Identity Service** — cadastro, login, perfil, credenciais, estado da conta, revogação e coordenação do encerramento.
- **Budget Service** — projetos, teto, itens/alternativas, lojas, alterações, estados e atualização controlada de status após oportunidade.
- **Monitoring Service** — snapshot de orçamento, coletor, orquestração IA/mock/fallback, registro de tentativas/oportunidades, consulta ao histórico e bloqueio de monitoramentos

após encerramento.

Todos operam em Azure Functions Node.js LTS, distribuídos em **Function Apps independentes**, com autonomia de implantação, variáveis de ambiente e credenciais próprias. A organização interna dos serviços segue as fatias verticais indicadas no UML de Componentes, sem chamadas diretas ao banco de outro serviço.

Persistência NoSQL e propriedade de dados

O cluster MongoDB Atlas hospeda os bancos lógicos `db_identity`, `db_budgets` e `db_monitoring`. Os dados referenciam IDs de outro contexto **apenas logicamente**, não como chaves estrangeiras verificadas entre bancos. O Budget incorpora itens e lojas em `orcamentos`; o Monitoring conserva **snapshots** dos dados necessários para executar a análise independentemente da disponibilidade momentânea do Budget. A sincronização ocorre por eventos; o versionamento impede substituição por snapshots mais antigos.

Mensageria e consistência eventual

Azure Storage Queues fornece três filas: **monitoramento**, **resultados** e **privacidade**. Não há *fan-out* automático: o Identity envia **duas cópias com destinos explícitos** do pedido de encerramento (para Monitoring e Budget). O Budget publica eventos de orçamento para monitoramento, e o Monitoring publica oportunidades na fila de resultados. Ambos confirmam a privacidade em mensagens individuais na fila de privacidade. Fila de monitoramento e fila de resultados podem transportar mais de um tipo de mensagem, portanto cada fila terá um **handler de despacho por tipo**, e não Queue Triggers separados competindo pela mesma mensagem.

Transactional outbox embutido: a mesma operação de escrita em um documento MongoDB persiste o estado e um registro de evento pendente. Um dispatcher com Timer Trigger lê registros pendentes, envia à fila e confirma o envio. Uma interrupção após enviar e antes de marcar como publicado pode gerar duplicação, exigindo consumidores idempotentes. Trata-se de garantia **ao menos uma vez**, não exatamente uma vez nem transação distribuída entre MongoDB e filas.

IA, mock e fallback

O `AIEngineAdapter` pode chamar Gemini externo ou, em demonstração, um mock local de respostas estruturadas. O *Circuit Breaker* protege exclusivamente a chamada externa Gemini; na indisponibilidade, JSON inválido, erro HTTP 429/5xx ou timeout, o `FallbackHeuristico` seleciona uma cesta candidata local. O `RegrasDeCompra` confere

disponibilidade, compatibilidade, itens obrigatórios e teto. O fallback **não garante ótimo global**, e sem ofertas suficientes a tentativa recebe motivo `SEM_DADOS`, sem criação de oportunidade.

4.2 Decomposição de Alto Nível e Padrões Arquiteturais

Tabela 9: Decomposição de alto nível da solução.

Elemento	Responsabilidade	Estilo / tecnologia
Web Shell	Composição, rotas e elementos globais do frontend.	Microfrontend / React / Vite
MFE Conta	Cadastro, login, perfil, encerramento.	Microfrontend / build independente
MFE Orçamentos	Projetos, itens, lojas, histórico, painel.	Microfrontend / build independente
Gateway/BFF	Ponto de entrada HTTP, JWT, composição e roteamento.	Gateway lógico/BFF / Functions HTTP
Identity Service	Gestão de identidade e coordenação de privacidade.	Microsserviço FaaS / db_identity
Budget Service	Regras de orçamento, estados e publicação de eventos.	Microsserviço FaaS / db_budgets
Monitoring Service	Varredura, avaliação, registros e resultados.	Microsserviço FaaS / db_monitoring
Azure Storage Queues	Transporte de eventos e trabalho assíncrono.	EDA / ao menos uma vez
Gemini / lojas	Inferência opcional e catálogos externos.	Ports & Adapters / HTTPS

Clean Architecture + Vertical Slice: cada serviço organiza casos de uso em fatias como `CriarOrcamento`, `AtualizarAssinatura`, `ConsultarOportunidades`, `ExecutarVarredura` ou `EncerrarContaMonitoring`. A camada de aplicação chama interfaces (`IOrcamentoRepository`, `IMonitoringRepository`, `ICatalogoPort`, `IAIEnginePort`, `IEventPublisher`); componentes de infraestrutura as implementam. Regras de compra não importam bibliotecas do Azure, SDK da LLM ou MongoDB. As Figuras 04, 11A, 11B e 12 sustentam esta decomposição.

4.3 Estratégias para Cumprimento das Metas de Qualidade

Tabela 10: Estratégias e mecanismos de qualidade.

Atributo	Mecanismos arquiteturais	Verificação de qualidade
Segurança/privacidade	bcrypt, JWT, estado atual, revogação, autorização interna, segregação do banco.	Testes de titularidade, login, conta encerrada e credenciais.
Resiliência	Timeout/Circuit Breaker, mock, fallback, tentativas com motivos.	Testes 429/5xx/timeout e compra incompleta sem alerta.
Consistência	Outbox, versões, idempotência, marcador de privacidade.	Reenvio/duplicação/ordem invertida de eventos e confirmação eventual.
Modificabilidade	MFEs independentes, Function Apps autônomas, interfaces e slices.	Pipeline independente, contrato API, testes do domínio sem rede.
Desempenho/custo	CDN, snapshots, Timer/Queue Triggers, pooling e limites de concorrência.	Testes de carga e orçamento de cotas em ambiente real.

4.4 Decisões Organizacionais e Processos de Engenharia

O processo de engenharia é **orientado a contratos**: eventos, APIs e módulos remotos possuem versionamento explícito, preservando a compatibilidade entre versões de orçamento, snapshots e consumidores. Cada unidade possui pipeline de verificação e publicação independente, com validação de interfaces antes da integração. Segredos permanecem fora do Git; ambientes locais utilizam substitutos controlados de IA, catálogos e filas para testes reproduzíveis. A documentação registra as decisões estruturais e relaciona cada comportamento aos diagramas correspondentes.

4.5 Matriz de Alinhamento Estratégico

Tabela 11: Decisões, atributos de qualidade e restrições.

Decisão	Atributos	Requisitos/restrições	Trade-off reconhecido
Web Shell + 2 MFEs	Modificabilidade, autonomia	RT-02, RO-05	Mais contratos e risco de incompatibilidade de versões.
Gateway/BFF lógico	Segurança, UX, controle de API	RT-03, RT-09	Salto HTTP adicional e maior responsabilidade no ponto de entrada.
3 Function Apps + bancos por serviço	Autonomia, isolamento	RT-01, RT-04	Mais implantação, credenciais, custo e observabilidade.
Storage Queues + outbox	Resiliência, disponibilidade	RT-06, RT-07	Consistência eventual e consumidor idempotente obrigatórios.
Mock + IA externa opcional + fallback	Disponibilidade e testabilidade	RT-11, RF12–RF13	Heurística não garante combinação ótima global.
Clean Architecture + Vertical Slice	Manutenibilidade	RC-01, RC-02	Sobrecarga inicial de interfaces para escopo pequeno.
Encerramento por eventos e confirmação	Segurança, privacidade	RT-10, RT-17	Demora eventual e necessidade de reconciliação.

5. Visão de Blocos de Construção

Esta visão estabelece a hierarquia de responsabilidade entre sistema, contêineres e componentes. Um contêiner C4 representa uma unidade de execução/implantação de software; caixas identificadas como **agrupamento lógico** organizam visualmente as unidades e não significam processos extras em execução.

5.1 Visão Sistêmica Geral de Caixa Branca (Referência ao C4 Nível 1)

A **Figura 02**, apresentada na seção 3.1, delimita o comprador e seus dois parceiros externos. A decomposição interna prioriza três capacidades de negócio: (i) identidade e privacidade; (ii) registro e governo do orçamento; (iii) monitoramento, sugestão e oportunidades. O Gateway/BFF integra essas capacidades sob um ponto lógico de entrada, enquanto eventos permitem que a experiência de cadastro/edição não aguarde uma varredura demorada.

5.2 Nível 2: Decomposição em Contêineres (C4 Nível 2)

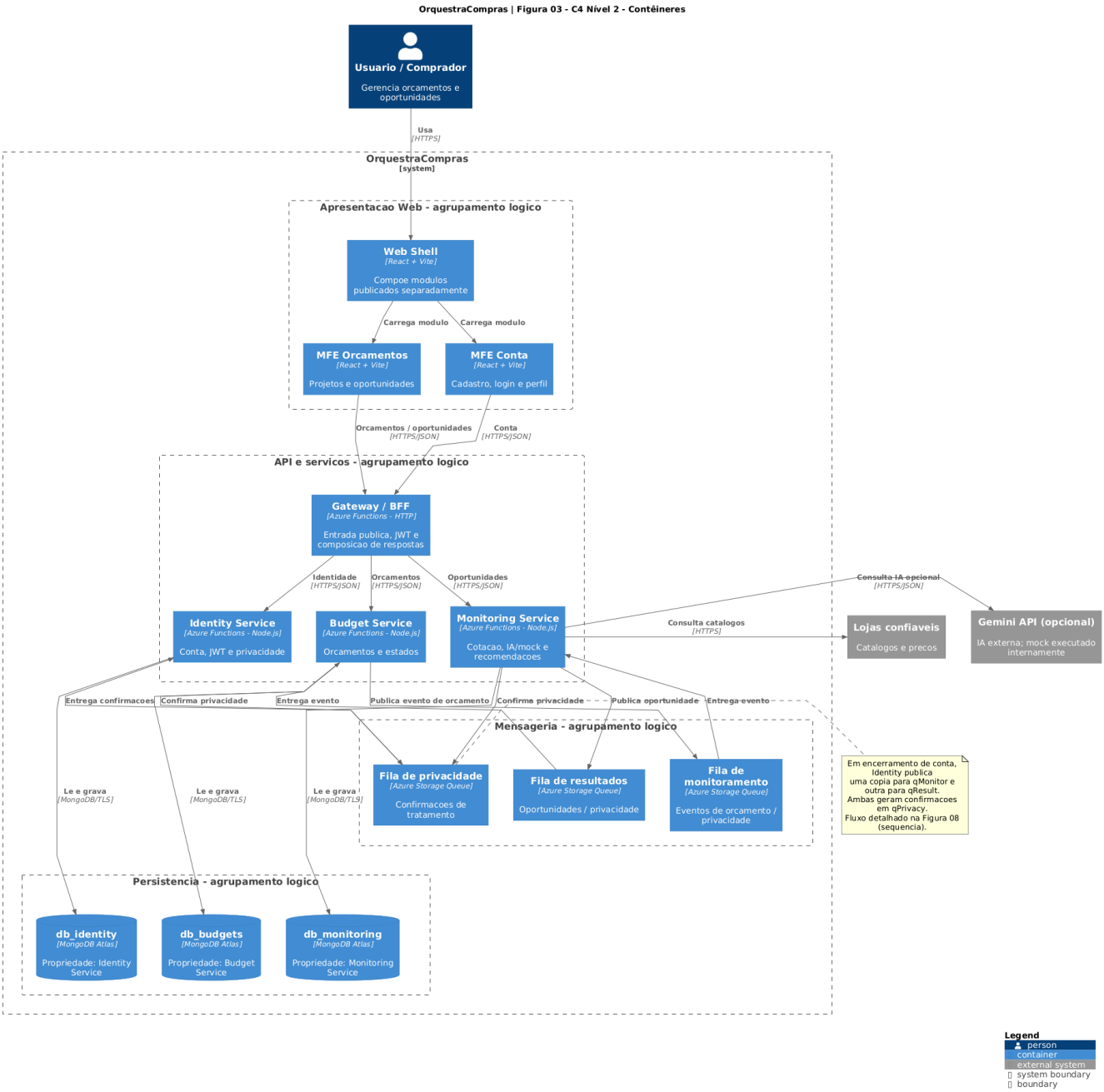


Figura 03: C4 Nível 2 — Contêineres e limites de responsabilidade da solução.

Tabela 12: Especificação dos contêineres e infraestrutura lógica.

Unidade	Tecnologia / execução	Dados / integração	Responsabilidade
Web Shell	React + Vite / publicação estática	Carrega MFEs	Host, rotas e layout global.
MFE Conta	React + Vite / publicação separada	HTTPS → BFF	Interfaces de identidade.
MFE Orçamentos	React + Vite / publicação separada	HTTPS → BFF	Orçamentos e resultados.
Gateway/BFF	Azure Functions HTTP / Function App	Identity, Budget, Monitoring	Autenticação/roteamento/composição.
Identity Service	Azure Functions / Function App	db_identity e filas	Usuário, JWT, privacidade.
Budget Service	Azure Functions / Function App	db_budgets; filas monitoring/resultados	Projetos, estados, outbox.
Monitoring Service	Azure Functions / Function App	db_monitoring, 3 filas, lojas/IA	Snapshots, varredura e oportunidades.
Azure Storage Queues	3 filas lógicas	monitoramento, resultados, privacidade	Mensagens assíncronas e tentativas de entrega.
MongoDB Atlas	3 bancos lógicos no cluster	db_identity, db_budgets, db_monitoring	Persistência privada de cada serviço.

Principais limites de propriedade

- O **Identity** é a única autoridade do status da conta e das credenciais; os outros serviços só conhecem `usuarioId` e estados de privacidade recebidos por contrato.
- O **Budget** é o único autorizado a alterar o orçamento. A confirmação de oportunidade recebida via evento é validada contra `orcamentoId`, versão, proprietário e estado atual, sem leitura de `db_monitoring`.
- O **Monitoring** não lê `db_budgets` nem `db_identity`; recebe **snapshots versionados** e mantém histórico local.
- As três filas são **recursos**, não microsserviços. O encaminhamento de mensagens a destinos distintos ocorre por publicação explícita dos produtores.

Razões do desacoplamento

Esta decomposição permite que uma consulta de oportunidade seja respondida sem acionar busca e IA em tempo real, que falhas de catálogo não bloqueiem cadastro e que o bloqueio de conta seja imediato para novas requisições, enquanto os efeitos nos demais serviços são reconciliados assincronamente.

5.3 Nível 3: Decomposição do Monitoring Service (C4 Nível 3)

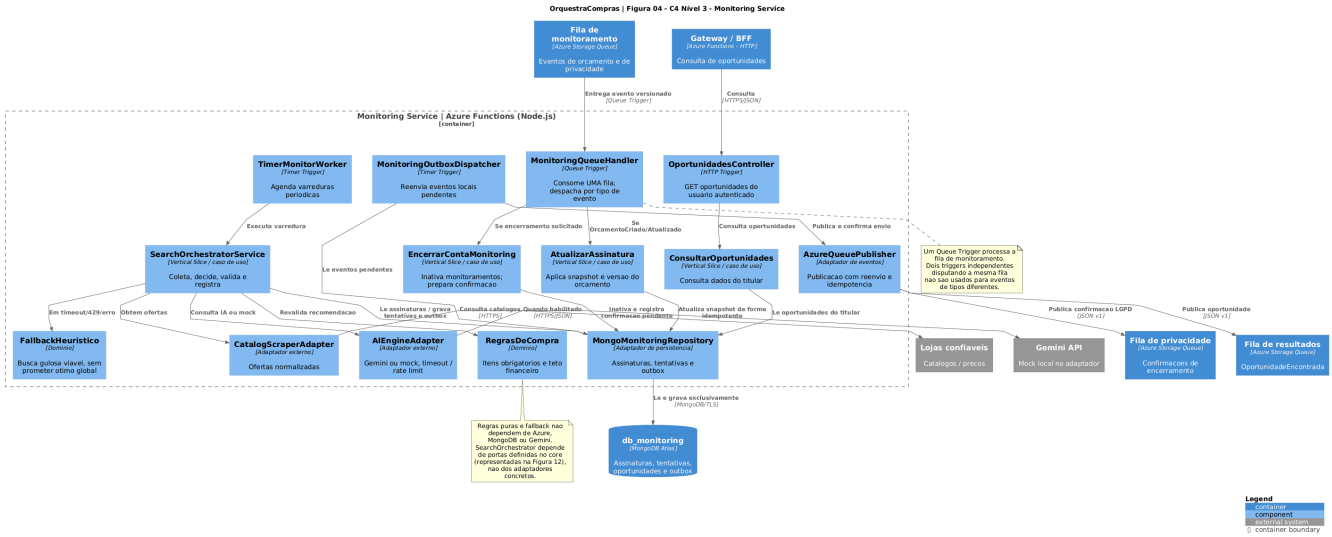


Figura 04: C4 Nível 3 — Componentes internos do Monitoring Service.

Tabela 13: Componentes do Monitoring Service e dependências.

Camada	Componente	Papel / saída
Entradas	TimerMonitorWorker	Agenda ciclos periódicos de varredura.
Entradas	MonitoringQueueHandler	Um Queue Trigger despacha eventos de orçamento ou privacidade por tipo.
Entradas	OportunidadesController	GET autenticado, lista oportunidades do titular.
Entradas	MonitoringOutboxDispatcher	Timer lê pendências locais e aciona publisher.
Aplicação / slices	SearchOrchestratorService	Coordena pesquisa e validação; registra resultado.
Aplicação / slices	AtualizarAssinatura	Aplica somente snapshots mais recentes e respeita marcador de encerramento.
Aplicação / slices	EncerrarContaMonitoring	Interrompe monitoramentos, trata dados e produz confirmação.
Aplicação / slices	ConsultarOportunidades	Filtra histórico por usuário e orçamento.
Domínio	RegrasDeCompra	Confere todos os itens, compatibilidade e orçamento.
Domínio	FallbackHeuristico	Produz candidato determinístico local sem garantia de ótimo.
Infraestrutura	MongoMonitoringRepository	Lê/grava assinaturas, tentativas, oportunidades, outbox e marcadores.
Infraestrutura	CatalogScraperAdapter	Consulta ou simula ofertas normalizadas.
Infraestrutura	AIEngineAdapter	Orquestra Gemini opcional ou mock interno.
Infraestrutura	AzureQueuePublisher	Publica oportunidades e confirmações com eventId.

Contratos e interfaces dos serviços

Tabela 14: Contratos HTTP e eventos da arquitetura.

Contrato	Origem → destino	Efeito / validação
POST /api/v1/auth/register	MFE Conta → BFF → Identity	Cria usuário, hash bcrypt, 201/409 conforme unicidade.
POST /api/v1/auth/login	MFE Conta → BFF → Identity	Autentica e emite JWT; recusa conta não ativa.
DELETE /api/v1/auth/account	MFE Conta → BFF → Identity	Aceita encerramento, revoga novos acessos, 202.
POST /api/v1/orcamentos	MFE Orçamentos → BFF → Budget	Valida e grava orçamento + EventoOrcamentoV1 pendente.
GET/PUT/DELETE /api/v1/orcamentos/{id}	MFE Orçamentos → BFF → Budget	Operações com titularidade, versão e estados.
GET /api/v1/oportunidades	MFE Orçamentos → BFF → Monitoring	Consulta filtrada por titular e orçamento.
EventoOrcamentoV1	Budget → fila monitoramento → Monitoring	Snapshot de orçamento versionado, upsert idempotente.
EventoOportunidadeV1	Monitoring → fila resultados → Budget	Resultado coerente com versão atual; Budget atualiza estado quando válido.
EventoContaEncerramentoSolicitadoV1	Identity → filas resultados e monitoramento	Cópias independentes destinadas a Budget/Monitoring.
PrivacidadeProcessadaV1	Budget e Monitoring → fila privacidade → Identity	Ack por serviço e operacaoId, finalização após ambas confirmações.

Governança de contratos: o Gateway/BFF disponibiliza a API pública sob `/api/v1/...`, com DTOs validados, autenticação e contratos estáveis para os microfrontends. Os serviços mantêm interfaces internas e mensagens JSON versionadas, evitando exposição de implementações privadas e possibilitando evolução independente por compatibilidade explícita.

6. Visão de Tempo de Execução

Os cenários descrevem o comportamento operacional do sistema por mensagens síncronas e assíncronas. As Figuras 05–08 representam o detalhamento comportamental do C4 Nível 4 mediante UML de Sequência. Os fluxos incluem tratamento de falhas, duplicidade e idempotência; retorno HTTP de aceite não equivale à conclusão do processamento em segundo plano.

6.1 Cenário 1: Ciclo Autônomo de Varredura e Avaliação por IA (Agente com Fallback)

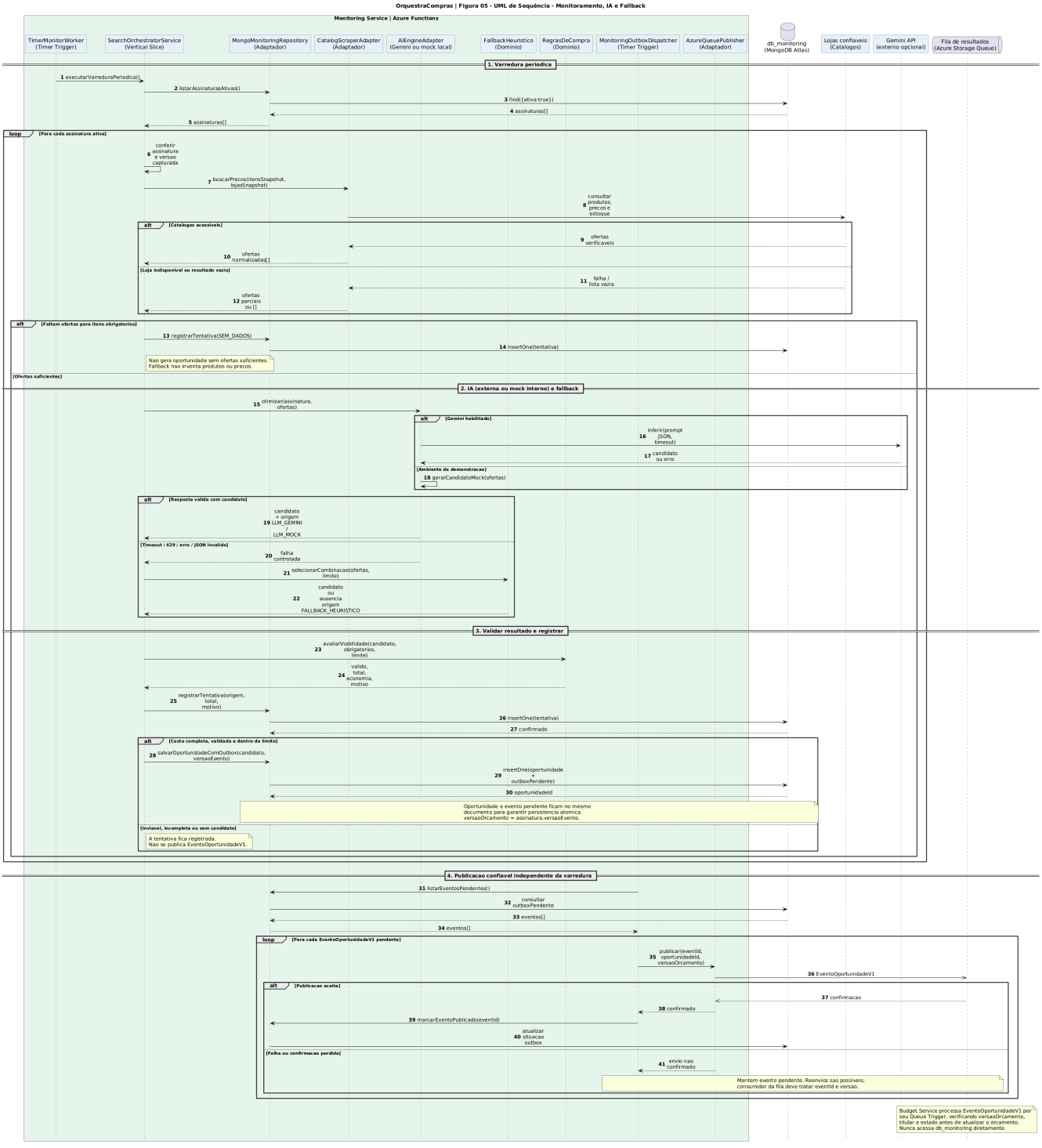


Figura 05: UML Sequência — Monitoramento, IA, mock e fallback heurístico.

Dinâmica das interações

1. `TimerMonitorWorker` inicia o ciclo; `SearchOrchestratorService` consulta `assinaturasMonitoramento` ativas em `db_monitoring` usando

`MongoMonitoringRepository`.

2. Para cada assinatura não encerrada, o `CatalogScraperAdapter` coleta ofertas normalizadas para `itensSnapshot` nas `lojasSnapshot`. Falhas pontuais de loja são isoladas e registradas.
3. Se faltam ofertas verificáveis para algum item obrigatório, registra-se a tentativa `SEM_DADOS` sem oportunidade. Não se simula um produto em estoque ou um preço como real sem identificá-lo como dado de teste.
4. Havendo dados suficientes, `AIEngineAdapter` seleciona Gemini externo (se habilitado) ou mock local. Timeout, 429, erro 5xx, JSON inválido ou falha do circuito acionam `FallbackHeuristico`.
5. O núcleo `RegrasDeCompra` recalcula total e verifica requisitos obrigatórios, compatibilidade, teto e versão de snapshot. Uma resposta da IA não pode invalidar a política de domínio.
6. Toda tentativa é registrada. Uma cesta **completa, válida e dentro do limite** gera `OportunidadeAlerta` e um `EventoOportunidadeV1` pendente no mesmo documento do agregado; caso contrário, não se publica oportunidade.
7. Em execução independente, `MonitoringOutboxDispatcher` publica os eventos pendentes na **fila de resultados** e marca a entrega aceita. Reenvio pode ocorrer e o Budget trata duplicações.

Tabela 15: Matriz de responsabilidades — cenário de monitoramento.

Etapa	Responsável	Entrada / saída	Tratamento de exceção
Inicialização	TimerMonitorWorker	Assinaturas ativas	Isola falhas por ciclo/assinatura.
Coleta	CatalogScraperAdapter	Catálogo → ofertas válidas	Falhas por loja; SEM_DADOS sem dados suficientes.
Sugestão	AIEngineAdapter	Candidato por Gemini ou mock	Timeout/Circuit Breaker/Fallback.
Validação	RegrasDeCompra	Candidato → válido/inválido	Não publica combinação incompleta.
Persistência	MongoMonitoringRepository	Tentativa/oportunidade + outbox	Escrita por agregado e controle de versão.
Publicação	MonitoringOutboxDispatcher	EventoOportunidadeV1	Retry + consumidor idempotente.

6.2 Cenário 2: Cadastro, Autenticação e Emissão de Token JWT

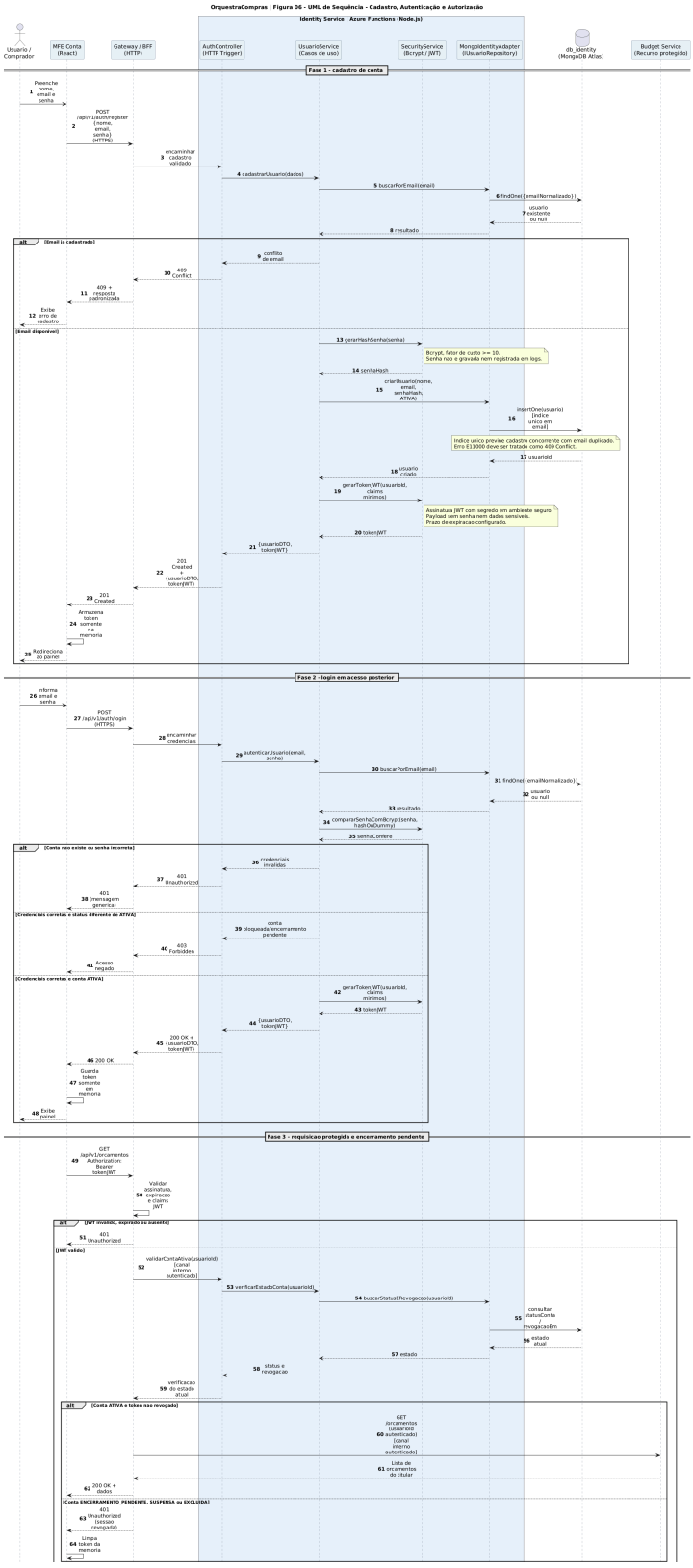


Figura 06: UML Sequência — Cadastro, autenticação JWT e bloqueio de acesso.

Dinâmica das interações

Cadastro: comprador envia `nome`, `email`, `senha` ao MFE Conta; o BFF valida transporte/payload e chama Identity. `UsuarioService` verifica formato e unicidade, gera hash por `SecurityService`, persiste usuário ativo em `db_identity`, emite JWT e devolve `201 Created`. O índice único sobre email normalizado é a autoridade final em cadastros concorrentes (E11000 → `409 Conflict`).

Login: Identity consulta usuário, compara bcrypt (com caminho seguro para email inexistente), nega credenciais erradas com `401` e conta inativa com `403`; para `ATIVA` emite JWT com prazo curto e dados mínimos.

Acesso: BFF verifica assinatura/validade do JWT e consulta o estado atual/revogação do Identity por **canal interno autenticado** antes de encaminhar ao Budget. O Budget ainda aplica autorização por proprietário do recurso. Se a conta entrou em `ENCERRAMENTO_PENDENTE`, `SUSPENSA` ou `EXCLUIDA`, novas operações são negadas; token local é descartado pela UI. A checagem de estado adiciona latência e exige estratégia de indisponibilidade segura (falhar fechado em rotas sensíveis).

Tabela 16: Matriz de responsabilidades — cadastro e JWT.

Fase	Responsável	Controle	Resposta
Registro	Identity/UsuarioService	Índice email, bcrypt, segredo seguro	201 ou 409
Login	Identity/SecurityService	Hash + estado ATIVA	200, 401 ou 403
Entrada	Gateway/BFF	JWT e estado em Identity	401/403 ou roteamento
Domínio	Budget ou Monitoring	Verifica titularidade de recurso	200/403/404 conforme contrato
Conta encerrando	Identity	Bloqueio/revogação	Negação imediata de novos acessos

6.3 Cenário 3: Criação de Orçamento e Sincronização por Eventos

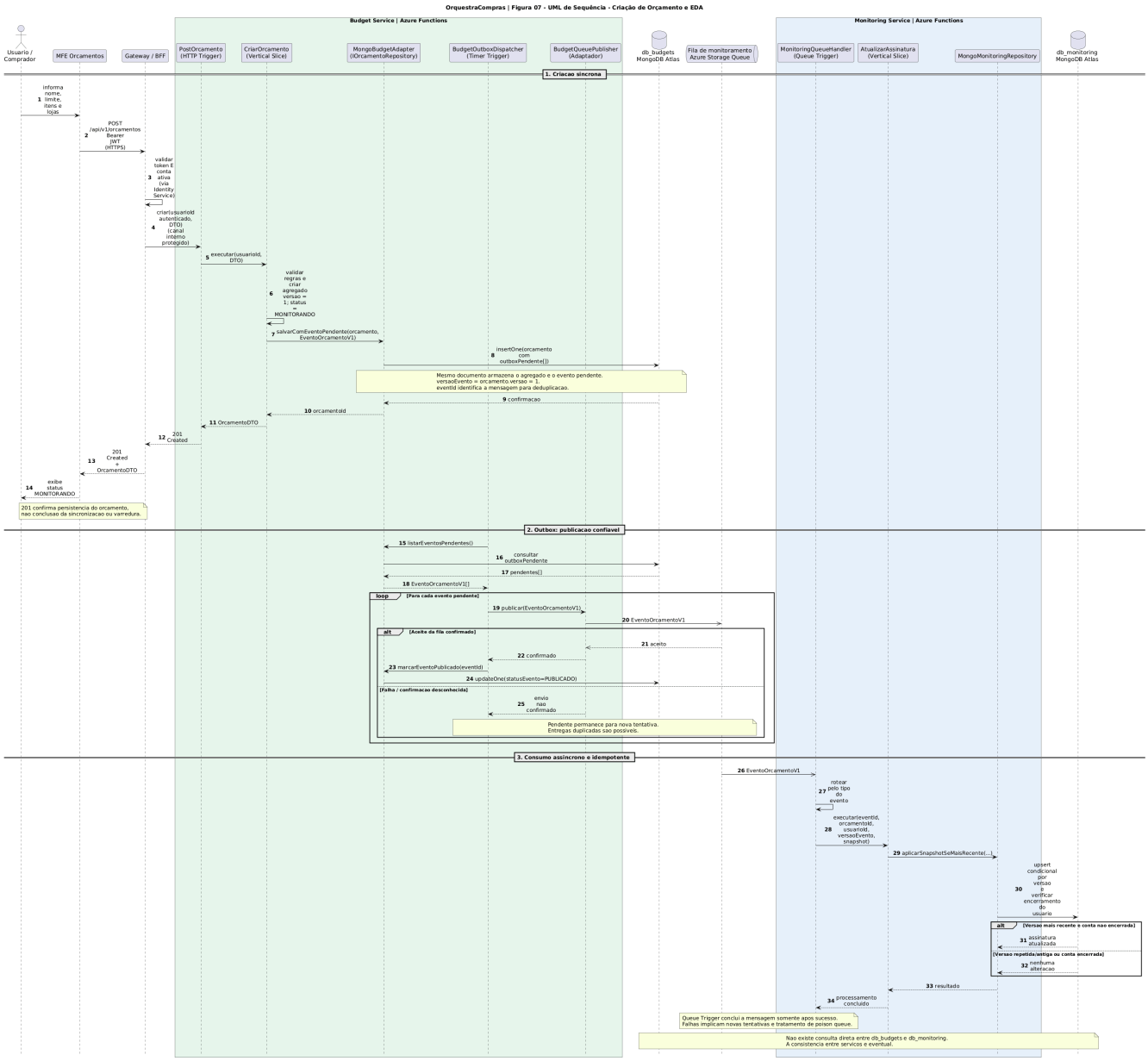


Figura 07: UML Sequência — Criação de orçamento e sincronização por eventos.

Dinâmica das interações

1. O MFE Orçamentos envia projeto, teto, itens e lojas ao BFF; o BFF autoriza o titular e encaminha ao `Budget Service`.
2. A fatia `CriarOrcamento` valida integridade e regras, cria `Orcamento` com `status=MONITORANDO` e `versao=1` e grava **orçamento + evento de outbox pendente na mesma atualização de documento** do MongoDB.
3. Retorna `201 Created` após persistência. O retorno **não garante** que o Monitoring já recebeu o orçamento.

4. `BudgetOutboxDispatcher` identifica evento pendente, publica `EventoOrcamentoV1` na **fila de monitoramento** e confirma envio aceito pela fila. Em caso de dúvida, conserva o evento para reprocessamento.
5. `MonitoringQueueHandler` retira a mensagem e despacha por tipo. `AtualizarAssinatura` verifica se `eventoId` já foi processado, se existe marcador de conta encerrada e se a `versao` recebida é mais recente que `versaoEvento` local. Se válida, atualiza o snapshot sem consultar `db_budgets`.
6. A confirmação da fila segue a conclusão segura do handler; erros transitórios provocam repetição e mensagens persistentes devem ser enviadas para tratamento (*poison queue*).

Tabela 17: Matriz de responsabilidades — criação e EDA.

Etapa	Componente	Consistência / garantia
Comando	PostOrcamento/CriarOrcamento	Síncrona até escrita no Budget; 201.
Persistência	MongoBudgetAdapter	Documento e outbox juntos.
Publicação	BudgetOutboxDispatcher/Publisher	Ao menos uma vez; possível duplicata.
Consumo	MonitoringQueueHandler	Um handler por fila, despacho por tipo.
Aplicação	AtualizarAssinatura	Idempotência, versão mais recente e marcador de encerramento.

6.4 Cenário 4: Encerramento de Conta e Tratamento de Privacidade (LGPD)

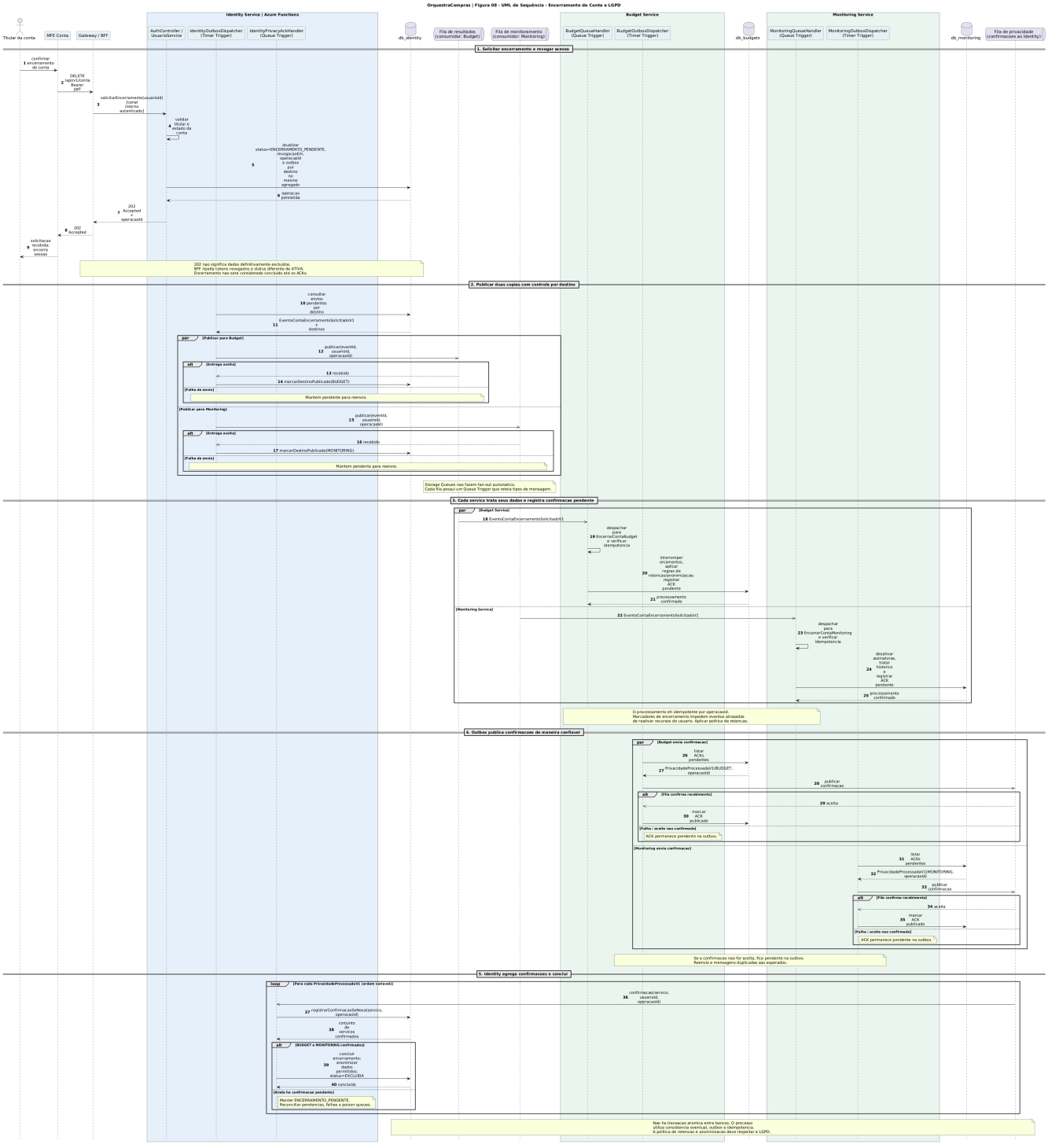


Figura 08: UML Sequência — Encerramento de conta e tratamento assíncrono de privacidade.

Dinâmica das interações

1. O titular confirma o encerramento. `Identity Service` autentica a operação e registra `statusConta=ENCERRAMENTO_PENDENTE`, `revogacaoEm` e evento pendente no mesmo agregado. O BFF responde `202 Accepted`, não `200` de exclusão completa.

2. `IdentityOutboxDispatcher` envia **duas cópias distintas** de `EventoContaEncerramentoSolicitadoV1` com o mesmo `operacaoId`: uma à fila de resultados para Budget e outra à fila de monitoramento para Monitoring. Azure Storage Queues não faz distribuição automática a dois consumidores.
3. Cada serviço consome a mensagem destinada a si, registra marcador `operacoesPrivacidade` local de forma idempotente, interrompe novos efeitos de negócio, trata registros conforme política de retenção e cria sua confirmação pendente **no seu próprio banco**.
4. Os dispatchers de Budget e Monitoring publicam `PrivacidadeProcessadaV1` na **fila de privacidade**; uma falha antes da confirmação de envio conserva a pendência. Após entrega, eventuais duplicatas são toleradas.
5. `IdentityPrivacyAckHandler` aplica confirmações por par `(operacaoId, servico)`; somente quando as confirmações exigidas foram registradas e não há pendências a resolver o Identity conclui o encerramento e executa anonimização/remoção conforme política aprovada.
6. `operacoesPrivacidade` em Budget e Monitoring impede que um `EventoOrcamentoV1` atrasado reative monitoramento ou recrie dados depois da solicitação. Não existe transação distribuída entre os três bancos; falhas exigem reenvio, reconciliação e monitoramento operacional.

Escopo jurídico: LGPD não estabelece um apagamento indiscriminado em prazo único. Retenção por obrigação legal, auditoria e prevenção a fraude deve ser decidida/documentada com base legal e prazo definidos. O desenho é mecanismo de suporte à política, não certificação automática de conformidade.

Tabela 18: Matriz de responsabilidades — encerramento distribuído.

Fase	Serviço	Compromisso / idempotência
Aceite	Identity	Bloqueio/revogação + outbox; 202.
Distribuição	Identity dispatcher	Publicação para dois destinos, rastreamento por operacaoId e destino.
Tratamento de dados	Budget	Marca conta encerrada, impede alterações, trata orçamentos e gera confirmação local.
Tratamento de dados	Monitoring	Desativa assinaturas, aplica retenção/anonimização, gera confirmação local.
Reconciliação	Identity privacy handler	ACK único por serviço; conclui somente com ambos + política satisfeita.

6.5 Aspectos Relevantes da Execução de Runtime

- **Ordem de eventos:** o consumidor ignora snapshots com `versaoEvento` menor ou igual à local, após confirmar a identidade do produtor e o tipo do contrato.

- **Duplicidade:** mensagens podem chegar mais de uma vez. `eventId`, `operacaoId` e controle local de efeito previnem dupla oportunidade ou reativação.
- **Falha de publicação:** outbox conserva evento não confirmado; publicadores e consumidores registram tentativas, métricas e *poison messages*.
- **Concorrência:** timer pode sobrepor execuções e filas podem distribuir mensagens; operações de estado devem ser condicionais/atômicas por documento.
- **Propriedade:** não há consulta cruzada de bancos; apenas HTTP autenticado ou eventos versionados.
- **Integridade comercial:** `RegrasDeCompra` impede a publicação de uma cesta incompleta ou acima do teto, inclusive quando mock/LLM erra.
- **Diagnóstico:** traces correlacionam HTTP inicial, comando, evento, assinatura e tentativa sem logar senha/JWT/PII.
- **Disponibilidade:** falhas de serviço Identity em validação de conta não autorizam o BFF a assumir estado ativo por padrão.
- **Conciliação:** consumidores executam rotinas de reprocessamento de eventos pendentes e comparação periódica de estados de privacidade.

7. Visão de Implantação

A topologia cloud distribui os componentes por unidades de implantação independentes: três publicações estáticas no frontend, quatro Function Apps no backend, filas do Azure Storage e três bancos lógicos no MongoDB Atlas. Os limites entre essas unidades determinam a autonomia de publicação, as permissões e os caminhos de comunicação.

7.1 Nível de Infraestrutura 1: Topologia Cloud e Ambientes

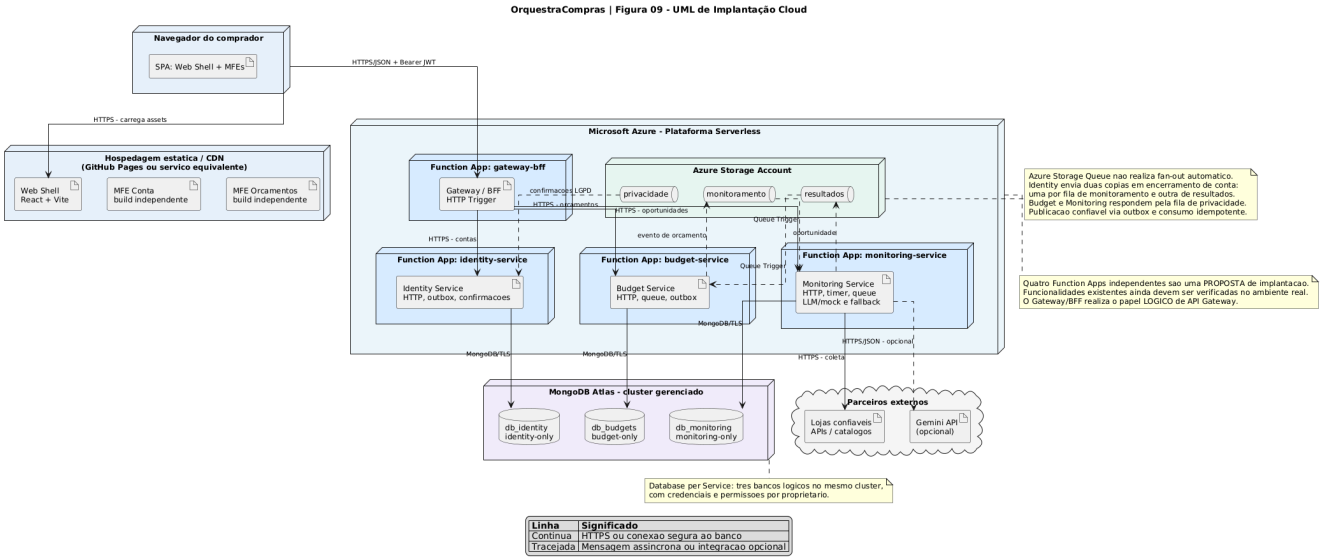


Figura 09: UML de Implantação Cloud — unidades independentes e infraestrutura gerenciada.

Nós de implantação e fronteiras de segurança

- **Navegador do comprador:** executa Web Shell + módulos remotos via HTTPS. Tokens não ficam embutidos em URLs de asset.
- **Hospedagem estática/CDN:** três distribuições estáticas (Shell, Conta, Orçamentos) com URLs publicadas, manifesto de módulos e CORS da API restrito às origens autorizadas.
- **Microsoft Azure:** Function Apps `gateway-bff`, `identity-service`, `budget-service` e `monitoring-service`; uma Storage Account com três filas do negócio e armazenamento necessário ao runtime Azure Functions. Prover privilégios independentes e mecanismos de autenticação interna adequados ao plano adotado.
- **MongoDB Atlas:** um cluster gerenciado com bancos `db_identity`, `db_budgets`, `db_monitoring`; usuários `identity-only`, `budget-only`, `monitoring-only` com autorização somente nos respectivos bancos.

- **Parceiros externos:** lojas confiáveis e Gemini API opcional. A presença do Gemini no diagrama não obriga seu uso em todos os ambientes.

Governança de custos: Azure Functions, Storage Queues, hospedagem e Atlas possuem cotas e preços dependentes de plano/região/conta. O custo operacional depende do volume de uso e dos limites contratuais de cada serviço; não decorre automaticamente da escolha serverless. O gerenciamento de consumo aplica alertas de orçamento e políticas de escalabilidade.

7.2 Mapeamento de Artefatos de Software para Nós de Infraestrutura

Tabela 19: Artefatos, nós de implantação e comunicação.

Artefato	Nó de implantação	Integração	Segurança e configuração
web-shell	Static site / CDN (publicação A)	Carrega MFEs	Manifest de versões e URLs seguras.
mfe-conta	Static site / CDN (publicação B)	BFF HTTPS	Nenhum segredo embutido no bundle.
mfe-orcamentos	Static site / CDN (publicação C)	BFF HTTPS	Nenhum segredo embutido no bundle.
gateway-bff	Azure Function App 1	Identity/Budget/Monitoring HTTP	JWT e credenciais internas sob configuração segura.
identity-service	Azure Function App 2	db_identity, monitoring/resultados, privacidade	JWT_SECRET e credenciais próprias.
budget-service	Azure Function App 3	db_budgets, filas monitoramento/resultados/privacidade	Usuário budget-only.
monitoring-service	Azure Function App 4	db_monitoring, filas, lojas, IA	Chave Gemini opcional, isolada.
Azure Storage Account	Serviço gerenciado	Filas monitoring/resultados/privacidade	Políticas de acesso por função, retry e poison queues.
MongoDB Atlas	Cluster compartilhado lógico	db_identity/budgets/monitoring	TLS, três usuários restritos, índices/limites de conexões.

Ambientes e operação: `local` (mocks e storage emulator quando compatível), `homologação` (integrações de teste) e `produção` (segredos isolados). Cada serviço tem build, deployment, migração de índices, rollback e smoke test próprios. Não usar o deploy conjunto dos três serviços como único critério de autonomia. Revisar logs e permissões após cada publicação.

8. Conceitos Transversais

Os conceitos transversais transformam os estilos arquiteturais da solução em políticas de persistência, segurança, modularidade, integração e operação aplicáveis a todos os serviços.

8.1 Modelo de Domínio e Persistência de Dados (DER e Dicionário NoSQL)

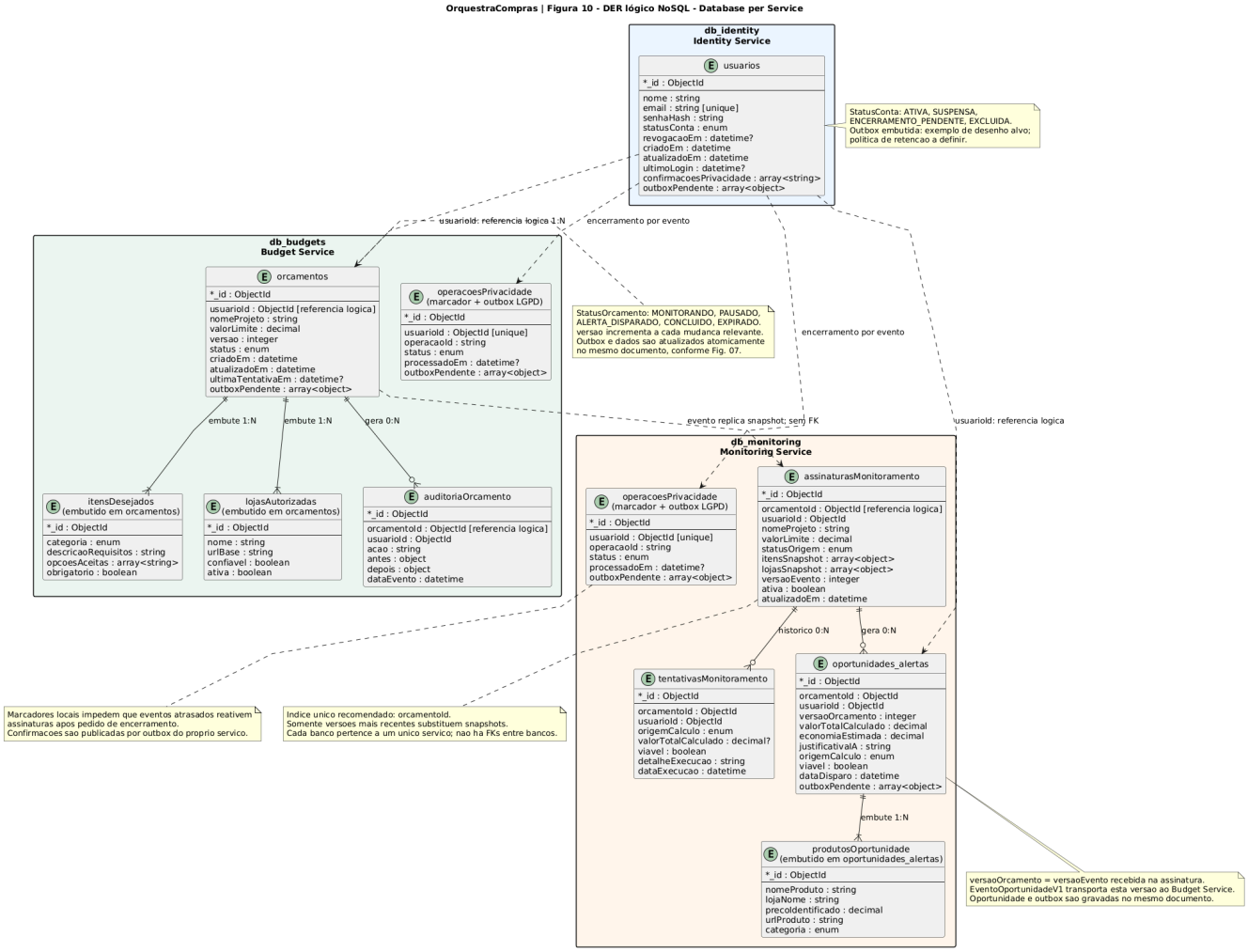


Figura 10: DER lógico NoSQL — Database per Service com snapshots e marcadores de privacidade.

8.1.1 Dicionário de Dados das Coleções e Documentos BSON

A Figura 10 apresenta um **DER lógico orientado a documentos**, sem FKs entre bancos. As referências `usuarioId` e `orcamentoId` são identificadores opacos de integração. Coleções e subdocumentos pertencem aos serviços proprietários, que aplicam validações JSON

Schema, índices e políticas de retenção dos dados pessoais. A separação por contexto mantém o isolamento da persistência e das credenciais.

Tabela 20: Dicionário NoSQL — db_identity.usuarios.

Campo	Tipo	Regra / responsabilidade
_id	ObjectId	Identificador local único.
nome	String	Dado pessoal, restrito ao Identity.
email	String	Normalizado e índice único; dado pessoal.
senhaHash	String	Hash bcrypt; nunca senha plana.
statusConta	Enum	ATIVA/SUSPENSA/ENCERRAMENTO_PENDENTE/EXCLUIDA.
revogacaoEm	DateTime?	Marco de revogação de sessões.
criadoEm / atualizadoEm / ultimoLogin	DateTime	Auditoria mínima.
confirmacoesPrivacidade	Array	Acks recebidos de Budget e Monitoring por operação.
outboxPendente	Array	Evento de encerramento e controle por destino.

Tabela 21: Dicionário NoSQL — db_budgets.orcamentos.

Campo	Tipo	Regra / responsabilidade
_id	ObjectId	Identificador do orçamento.
usuarioId	ObjectId lógico	Titular, não FK física.
nomeProjeto	String	Identificação exibida ao comprador.
valorLimite	Decimal	Teto positivo, não usar ponto flutuante binário para dinheiro.
versao	Integer	Incrementa em alteração relevante; base para snapshots.
status	Enum	MONITORANDO/PAUSADO/ALERTA_DISPARADO/CONCLUIDO/EXPIRADO.
itensDesejados	Array	Embutido com categoria, requisitos, opções, obrigatório.
lojasAutorizadas	Array	Embutido com nome, urlBase, confiavel, ativa.
criadoEm / atualizadoEm / ultimaTentativaEm	DateTime	Controle operacional.
outboxPendente	Array	Eventos versionados de mudança de orçamento.

Tabela 22: Dicionário NoSQL — subdocumentos e auditoria Budget.

Objeto	Campos essenciais	Observações
ItemDesejado	_id, categoria, descricaoRequisitos, opcoesAceitas[], obrigatorio	Embutido em orcamentos.
LojaAutorizada	_id, nome, urlBase, confiavel, ativa	Embutido em orcamentos; somente fontes permitidas.
auditoriaOrcamento	_id, orcamentoId, usuarioId, acao, antes, depois, dataEvento	Coleção do Budget; sanitizar PII e definir retenção.
operacoesPrivacidade (Budget)	_id, usuarioId único, operacaoId, status, processadoEm, outboxPendente	Marcador e confirmação por operação; barra eventos tardios.

Tabela 23: Dicionário NoSQL — db_monitoring.assinaturasMonitoramento.

Campo	Tipo	Regra / responsabilidade
_id	ObjectId	Identificador local.
orcamentoId	ObjectId lógico	Referência sem consulta cruzada.
usuarioId	ObjectId lógico	Contexto do titular.

Campo	Tipo	Regra / responsabilidade
nomeProjeto	String	Snapshot de dados não sensíveis necessários.
valorLimite	Decimal	Teto do snapshot recebido.
statusOrigem	Enum	Estado replicado de Budget.
itensSnapshot / lojasSnapshot	Array	Dados versionados; não lê db_budgets.
versaoEvento	Integer	Não aplicar versão antiga/duplicada.
ativa	Boolean	Interrompe monitoramento em privacidade.
atualizadoEm	DateTime	Controle de atualização.

Tabela 24: Dicionário NoSQL — tentativas, oportunidades e privacidade Monitoring.

Coleção/objeto	Campos essenciais	Invariante
tentativasMonitoramento	_id, orcamentoId, usuarioId, origemCalculo, valorTotalCalculado?, viavel, detalheExecucao, dataExecucao	Tentativas registradas mesmo sem oportunidade.
oportunidades_alertas	_id, orcamentoId, usuarioId, versaoOrcamento, valorTotalCalculado, economiaEstimada, justificativaIA, origemCalculo, viavel, dataDisparo, outboxPendente	Gerar apenas para cesta completa e financeiramente válida.
ProdutoIdentificado (embutido)	_id, nomeProduto, lojaNome, precoIdentificado, urlProduto, categoria	Produto da oportunidade; URL e preço precisam de fonte verificável.
operacoesPrivacidade (Monitoring)	_id, usuarioId único, operacaoId, status, processadoEm, outboxPendente	Tombstone impede reativação por eventos atrasados.

8.1.2 Validação de Esquema, Índices e Atomicidade

- **Unicidade:** índice `usuarios.email` único (normalizado); `orcamentos` com índices `(usuarioId, status)`, e histórico por `(usuarioId, orcamentoId, dataDisparo)`.
- **Snapshots:** índice único em `assinaturasMonitoramento.orcamentoId`; atualização condicional onde `versaoEvento` recebida seja maior e não exista marcador de encerramento.
- **Operação de privacidade:** unicidade por `usuarioId` / `operacaoId` conforme a modelagem escolhida; confirmar `PrivacidadeProcessadaV1` uma vez logicamente por operação e serviço.
- **Outbox:** anexar evento pendente à mesma alteração do documento de orçamento, oportunidade ou marcador. Publicador deve processar pendências sem crescimento ilimitado do documento; rotação e histórico exigem política própria.
- **Limites do modelo documental:** a escrita de um documento é atômica; **não** há atomicidade automática entre coleções diferentes nem entre MongoDB e Azure Storage Queues. Por isso, o estado de `operacoesPrivacidade` e o evento de saída correspondente integram a mesma unidade documental quando se exige persistência atômica; publicações externas ocorrem posteriormente por dispatcher, com idempotência e reconciliação.

- **Valores financeiros:** usar Decimal128/serialização decimal estável, arredondamento consistente e validação de origem dos preços; cálculos e testes não dependem de `float` impreciso.
- **Risco de retenção:** histórico e logs não são apagados arbitrariamente por TTL sem aprovação da política de privacidade e das obrigações legais.

O DER apresenta o **modelo lógico NoSQL**, incluindo limites de propriedade, referências entre contextos, subdocumentos embutidos e índices de integridade. As regras de atomicidade aplicam-se a documentos do mesmo agregado, sem pressupor transações globais entre serviços.

8.2 Padrões de Projeto e Estrutura Estática (Diagrama de Classes UML)

A estrutura estática é apresentada em duas perspectivas complementares: a **Figura 11A** detalha Identity, Budget e os contratos de eventos; a **Figura 11B** apresenta entidades, objetos de valor e regras do Monitoring Service. Os modelos utilizam classes e interfaces organizadas por contexto delimitado, sem dependência direta entre domínios.

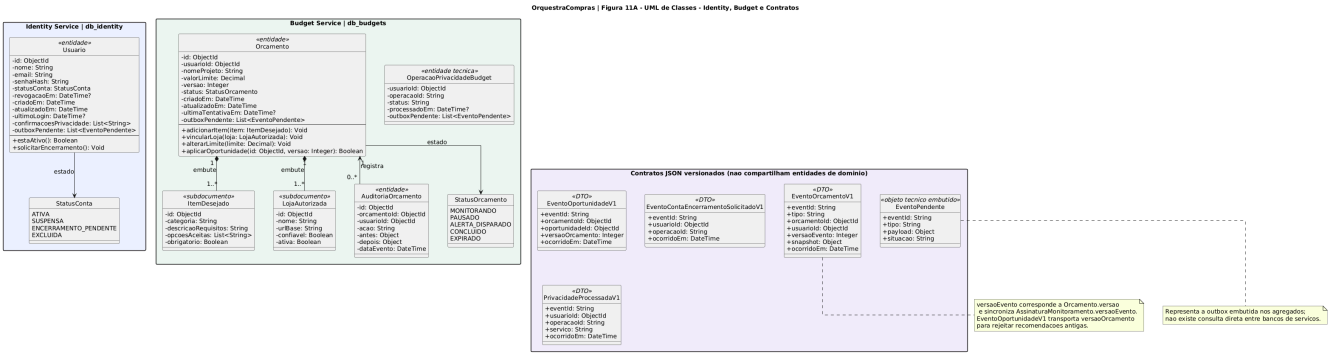


Figura 11A: UML de Classes — Identity Service, Budget Service e contratos JSON versionados.

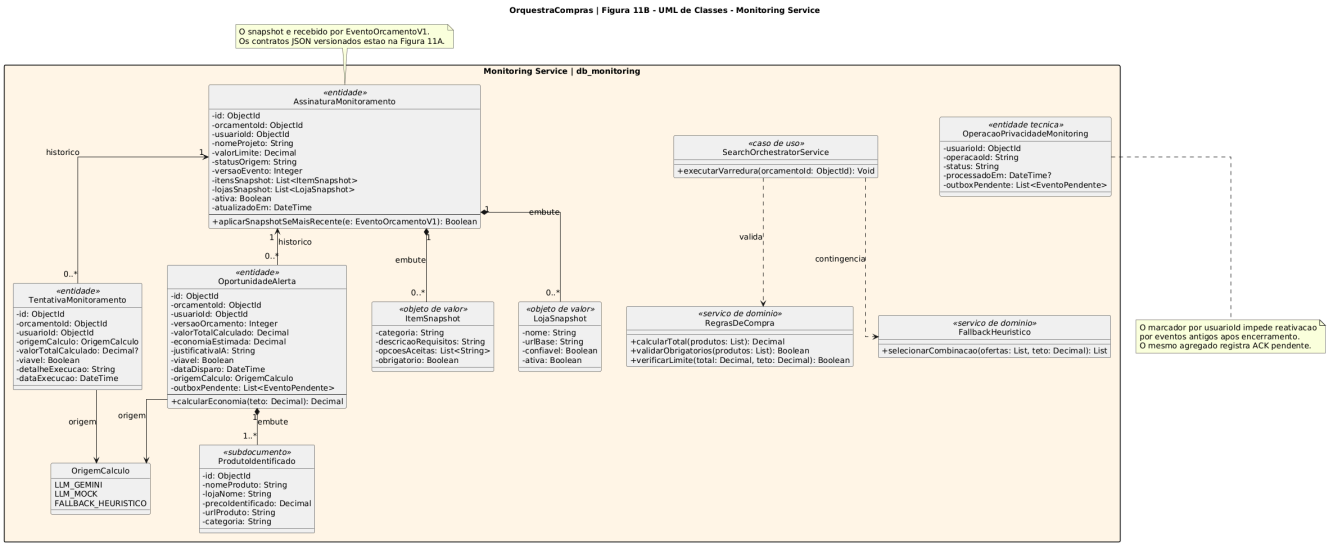


Figura 11B: UML de Classes — Domínio do Monitoring Service.

Tabela 25: Elementos do domínio, objetos de valor e serviços.

Contexto	Elemento	Invariantes / responsabilidade
Identity	Usuario / StatusConta	Credenciais, ciclo de vida, estado ATIVA → ENCERRAMENTO_PENDENTE → EXCLUIDA quando permitido.
Identity	Operações de privacidade	Confirmações por operaçãoId, auditoria e revogação de acesso.
Budget	Orcamento / StatusOrcamento	Raiz de agregado com teto, itens e lojas, versão e transições de estado.
Budget	ItemDesejado / LojaAutorizada	Subdocumentos que definem alternativas e fontes permitidas.
Budget	OperacaoPrivacidadeBudget	Tombstone e outbox de confirmação.
Monitoring	AssinaturaMonitoramento	Snapshot versionado e estado de monitoramento.
Monitoring	TentativaMonitoramento	Histórico e origem de cálculo sem obrigação de criar oportunidade.
Monitoring	OportunidadeAlerta / ProdutoIdentificado	Cesta completa, total, versão do orçamento e outbox.
Monitoring	RegrasDeCompra	Recalcula valor, compatibilidade e obrigatoriedade.
Monitoring	FallbackHeuristico	Candidato determinístico, sem prova de ótimo global.
Integração	DTOs de evento V1	Contrato JSON isolado do modelo de domínio; eventId e correlação.

Relacionamentos entre serviços: setas tracejadas representam contratos e eventos, **não** importação de classes de domínio entre microsserviços nem referências físicas entre bancos. A identidade do dono de orçamento (`usuarioId`) e as regras de versionamento atravessam fronteiras via contratos tipados e autorizados.

8.3 Modularização Física e Camadas de Código (Diagrama de Componentes UML)

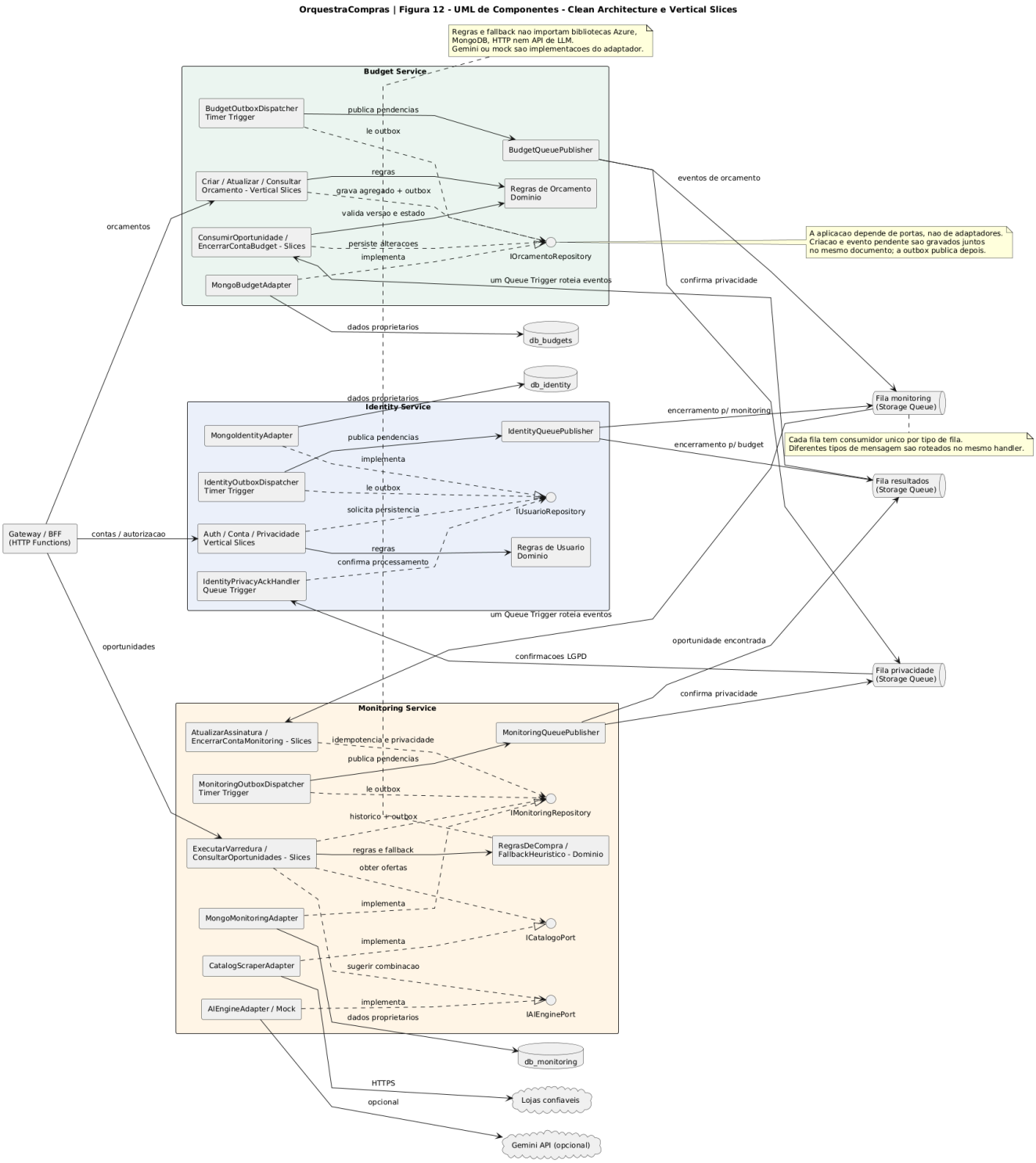


Figura 12: UML de Componentes — Clean Architecture, Vertical Slice e adaptadores.

A Figura 12 divide cada serviço em **entradas**, **slices/casos de uso**, **domínio**, **portas** e **adaptadores**. O contrato/porta permanece no núcleo e a implementação concreta aponta para ele: `MongoBudgetAdapter` implementa `IOrcamentoRepository`; `MongoMonitoringAdapter` implementa `IMonitoringRepository`; `AIEngineAdapter` e `CatalogScraperAdapter`

implementam as portas de IA e catálogo. Dispatchers leem a outbox por repositório próprio e publicam pelo adaptador Azure.

Organização física dos módulos

```
apps/
  web-shell/
  mfe-conta/
  mfe-orcamentos/
services/
  gateway-bff/
  identity-service/
    src/features/{register,login,encerrar-conta}/
    src/domain/  src/ports/  src/adapters/
  budget-service/
    src/features/{criar,atualizar,consultar,encerrar-conta}/
    src/domain/  src/ports/  src/adapters/
  monitoring-service/
    src/features/{atualizar-assinatura,executar-varredura,
                  consultar-oportunidades,encerrar-conta}/
    src/domain/  src/ports/  src/adapters/
contracts/
  events/v1/
infra/
  azure/  mongo/  local-mocks/
```

A organização distribui os módulos por aplicações e serviços, com fatias funcionais no diretório `features`, entidades e políticas em `domain`, interfaces em `ports` e implementações técnicas em `adapters`. `contracts/events/v1` reúne apenas schemas e DTOs públicos; as regras de domínio permanecem privadas de cada serviço. Contratos versionados permitem que os serviços sejam publicados de forma autônoma.

8.4 Diretrizes de Interface e Experiência do Usuário (UX/UI)

Tabela 26: Diretrizes UX/UI por fluxo.

Módulo	Tela ou função	Diretriz de interface
Web Shell	Layout e navegação	Rotas estáveis, carregamento de MFEs com fallback visual em erro de asset.
MFE Conta	Cadastro, login e perfil	Feedback de erros sem expor existência de conta além do contrato aprovado; acessibilidade.
MFE Conta	Encerramento de conta	Confirmação explícita, exibição de solicitação pendente (202) e status final separado.
MFE Orçamentos	Editor de projeto	Teto, categorias, alternativas, lojas e validações claras.
MFE Orçamentos	Painel de oportunidades	Mostrar total e fonte dos preços, economia, atualização e versão do resultado.
MFE Orçamentos	Estados de processamento	Distinguir MONITORANDO, ausência de dados, oportunidade válida e erro temporário.

Princípios de acessibilidade: navegação por teclado, rótulos associados a campos, feedback por texto além da cor, contraste adequado e tratamento de timeout. Os fluxos de conta, orçamento, painel e encerramento compartilham convenções de navegação, erros e estados assíncronos.

8.5 Conceitos de Segurança, Criptografia e Proteção da Informação

Tabela 27: Mecanismos de segurança e riscos tratados.

Mecanismo	Ameaça	Controle / teste
bcrypt	Vazamento de senhas	Hash único com salt, custo configurável, sem senha em logs.
JWT + checagem de status	Sessão com conta encerrada	Expiração, assinatura, revogação e consulta atual ao Identity.
Autorização de proprietário	Acesso horizontal/IDOR	Verificação de usuarioId para cada orçamento/oportunidade.
Autenticação de serviço	Falsificação de chamada interna	Credenciais por serviço, TLS e autorização de operação.
Segredos em ambiente	Exposição via frontend/Git	Sem secrets em bundles, rotação e detecção em CI.
MongoDB least privilege	Acesso cruzado entre bancos	Três contas técnicas restritas e auditoria de permissões.
Schemas e entradas	NoSQL injection/prompt injection	Allowlist de campos, validação de tipos, tratar catálogo/LLM como dados não confiáveis.
LGPD distribuída	Dados residuais e reativação	Marcas de privacidade, retenção definida, confirmação e reconciliação.

Prompt injection: títulos, descrições e dados coletados de lojas e a saída de LLM são **entradas não confiáveis**. O pipeline nunca executa instruções contidas nesses dados, nunca delega autorização nem consulta dados pessoais com base em conteúdo de prompts; `RegrasDeCompra` faz validação estrita antes de persistir uma oportunidade. O processo também exige atualização periódica dos contratos de JSON e limites de tamanho.

8.6 Resiliência Cognitiva: Circuit Breaker e Fallback Heurístico Local

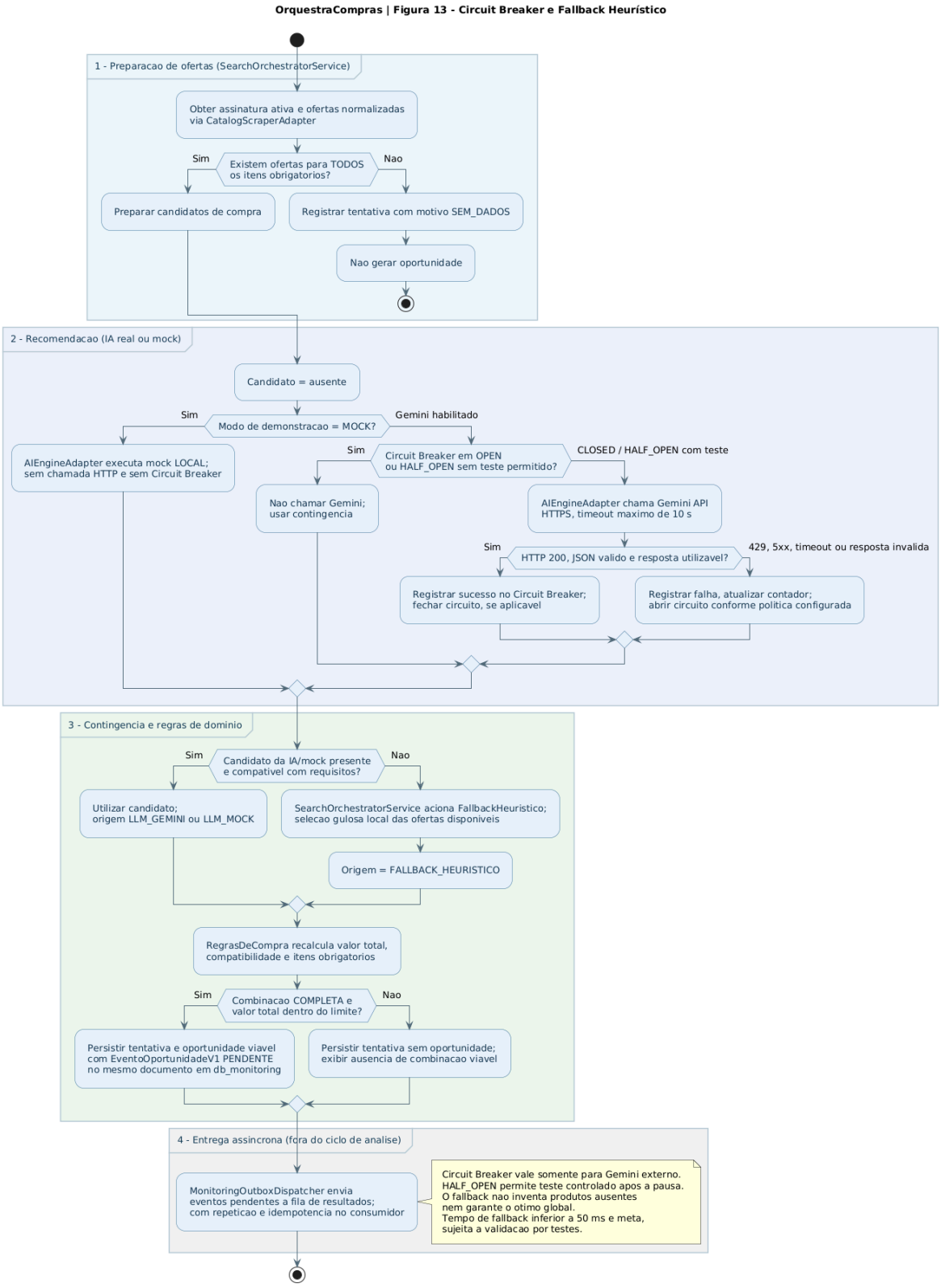


Figura 13: Fluxo de Circuit Breaker e Fallback Heurístico — decisão controlada pelo domínio.

Os estados do circuito externo são **CLOSED** (chamada permitida e monitorada), **OPEN** (chamada bloqueada, usa contingência) e **HALF_OPEN** (teste controlado de recuperação).

Resposta de erro 429, 5xx, timeout e saída inválida incrementam a política de falha configurada. O mock local bypassa a API externa e não é sinal de disponibilidade do Gemini. Falhas de scraping podem resultar em `SEM_DADOS` sem oportunidade; não podem ser encobertas por criação de dados fictícios.

O `FallbackHeuristico` escolhe ofertas disponíveis por critérios determinísticos definidos, mas não promete a combinação ótima global. O `RegrasDeCompra` precisa conferir **itens obrigatórios**, **compatibilidade**, **valor total**, **valor limite** e **origem verificável da oferta**. O tempo de execução inferior a 50 ms é um **objetivo de desempenho** do fallback local, cuja aferição depende das condições de carga e tamanho dos catálogos. Timeout externo previsto de 10 s também depende de configuração e teste em ambiente real.

8.7 Tratamento Padronizado de Erros e Validações HTTP

Tabela 28: Respostas HTTP e decisões de negócio.

Código / situação	Origem	Comportamento do contrato
201 Created	Identity / Budget	Novo usuário/orçamento persistido; processamento EDA é assíncrono.
202 Accepted	Identity	Encerramento aceito, sem declarar exclusão concluída.
400 Bad Request	BFF / serviço	Payload inválido, campo obrigatório ausente, valorLimite inválido.
401 Unauthorized	BFF / Identity	JWT ausente, inválido, expirado ou sessão revogada.
403 Forbidden	Serviço proprietário	Conta bloqueada, acesso a recurso de outro titular.
404 Not Found	Serviço proprietário	Recurso inexistente ou não visível segundo política.
409 Conflict	Identity / Budget	E-mail duplicado ou conflito de versão em concorrência.
429 Too Many Requests	Provedor externo	Circuit Breaker e contingência; não repassar necessariamente ao cliente.
500/502/503	Serviços / Gateway	Erro interno ou dependência; correlationId, sem stack trace exposto.

Envelope ilustrativo da resposta de erro:

```
{"timestamp":"2026-10-12T20:00:00Z","code":"ORCAMENTO_INVALIDO",
"message":"Informe um limite positivo.",
"correlationId":"corr-exemplo"}
```

O identificador de correlação é gerado na requisição, propagado de modo sanitizado nos eventos e não deve conter nome, e-mail, senha ou token.

8.8 Conceitos de Observabilidade e Telemetria

A observabilidade utiliza logs estruturados por função e evento com `service`, `operation`, `correlationId`, `eventId`, `orcamentoId`, versão, duração e resultado. Os painéis operacionais consolidam backlog e idade das filas, falhas por tipo, outbox pendente, duplicidade, tempo de

bloqueio de conta, duração de varredura, timeouts da IA e consumo de recursos. A retenção de telemetria limita custos e exposição de dados pessoais.

8.9 Conceitos de Testabilidade e Qualidade de Código

Tabela 29: Plano de teste por nível e evidência.

Nível	Cenário de teste	Critério de aprovação
Unitário domínio	RegrasDeCompra e limites Decimal	Nenhum alerta com item obrigatório ausente ou total acima do teto.
Unitário IA/mock	JSON inválido, timeout, 429, OPEN/HALF_OPEN	Fallback/local ou SEM_DADOS, sem travar lote.
Contrato HTTP	JWT, titularidade e schemas /api/v1	Status e corpo padronizados; sem acesso cruzado.
Contrato EDA	Campos V1, eventId, ordenação e versão	Consumidor rejeita antigo/duplicado sem efeitos repetidos.
Integração Mongo	Outbox embutida e índices	Escrita de estado+evento na unidade documental correta.
Integração Azure Queue	Retry e poison queue	Sem perda silenciosa e sem duplicação de oportunidades.
End-to-end	Cadastro → orçamento → snapshot → oportunidade	Caminho completo com mocks e dados de teste.
LGPD	Encerramento, ACKs fora de ordem e evento tardio	Acesso bloqueado; conclusão somente após duas confirmações.
Implantação	Deploy independente de MFE/Function App	Uma publicação não exige deploy conjunto de outros serviços.

Política de cobertura: regras financeiras, autorização, idempotência e transições de estado recebem prioridade nas suítes unitárias e de integração. Os relatórios de CI registram cobertura e cenários críticos, sem usar métricas presumidas.

8.10 Conceitos de Build, Versionamento e Entrega Contínua (CI/CD)

A esteira de CI/CD executa lint, testes unitários, validação de contratos e build individual de cada unidade. Os microfrontends e Function Apps são publicados independentemente, com *smoke tests* nas rotas públicas e nos casos de uso, rollback controlado e compatibilidade de contratos. Os artefatos frontend utilizam URLs remotas versionadas e o Web Shell gerencia cache e manifesto de versões. Segredos não são versionados e são fornecidos pelo ambiente de execução.

8.11 Resumo dos Conceitos Transversais

Tabela 30: Resumo dos conceitos transversais.

Conceito	Aplicação no OrquestraCompras	Evidência visual / seção
Isolamento de dados	Três bancos e três usuários restritos	Figuras 03, 09 e 10
Eventos confiáveis	Outbox embutida, dispatcher, at-least-once	Figuras 05, 07, 08 e 12
Integridade da compra	RegrasDeCompra + versão	Figuras 05, 11B e 13
Privacidade eventual	Tombstones, dupla confirmação	Figuras 08 e 10
Modularidade	Clean Architecture e Vertical Slice	Figuras 04, 11A/11B e 12
Frontend independente	Web Shell + MFEs autônomos	Figuras 03 e 09
Segurança ponta a ponta	JWT, escopos e validação em serviço	Figuras 06 e 08

9. Decisões Arquiteturais (ADRs)

As decisões arquiteturais a seguir registram contexto, alternativas consideradas, escolha técnica e consequências para a plataforma. Todas possuem status **Aceita**, com rastreabilidade para os requisitos e diagramas. Os ADRs integram a especificação e documentam as consequências das escolhas técnicas.

ADR 01 — Adoção de Microsoft Azure Functions como FaaS

- **Status:** Aceita.
- **Contexto do problema:** Evitar servidores permanentes, executar HTTP, Timer e Queue Triggers no mesmo ecossistema.
- **Alternativas consideradas:** VM/App Service; hospedagem monolítica persistente.
- **Decisão:** Quatro Function Apps independentes: gateway-bff, identity-service, budget-service e monitoring-service.
- **Justificativa, vantagens e consequências:** Escala por evento, mas cold starts, cotas, configuração e diagnóstico distribuídos.
- **Rastreabilidade:** RT-01, RT-13; Fig. 09.

ADR 02 — MongoDB Atlas como banco de documentos

- **Status:** Aceita.
- **Contexto do problema:** Orçamentos têm itens e lojas heterogêneos, enquanto histórico cresce.
- **Alternativas consideradas:** SQL relacional; um único banco NoSQL compartilhado.
- **Decisão:** MongoDB Atlas com bancos lógicos por serviço; itens/lojas embutidos no orçamento.
- **Justificativa, vantagens e consequências:** Agregados com escrita documental, limites de tamanho e controles de índices/retenção.
- **Rastreabilidade:** RT-04, RT-05; Fig. 10.

ADR 03 — Ports & Adapters e fallback para decisões cognitivas

- **Status:** Aceita.

- **Contexto do problema:** Gemini gratuito pode sofrer timeout, quota e JSON inválido.
- **Alternativas consideradas:** Depender sempre da LLM; calcular integralmente no controlador.
- **Decisão:** Separar IAIEnginePort, ICatalogoPort, mock local, fallback e regras de domínio.
- **Justificativa, vantagens e consequências:** Resiliência/testes melhores; combinação heurística não prova ótimo global.
- **Rastreabilidade:** RF12, RF13; Fig. 13.

ADR 04 — SPA composta por microfrontends

- **Status:** Aceita.
- **Contexto do problema:** Exigência de autonomia de frontend e atualização por domínio.
- **Alternativas consideradas:** SPA React monolítica; páginas renderizadas no servidor.
- **Decisão:** Web Shell + MFE Conta + MFE Orçamentos com build/deploy independente.
- **Justificativa, vantagens e consequências:** Novos contratos de roteamento, versões remotas e controle de cache.
- **Rastreabilidade:** RT-02; Fig. 03 e 09.

ADR 05 — JWT com bcrypt e consulta de estado de conta

- **Status:** Aceita.
- **Contexto do problema:** Tokens stateless não são revogados apenas por alteração do banco.
- **Alternativas consideradas:** Sessão persistida; JWT sem verificação de estado.
- **Decisão:** JWT curto e assinado, bcrypt para hash e checagem atual de conta no Identity.
- **Justificativa, vantagens e consequências:** Custo de consulta adicional e política fail-closed em indisponibilidade.
- **Rastreabilidade:** RT-09; Fig. 06.

ADR 06 — Gateway/BFF como porta lógica única

- **Status:** Aceita.
- **Contexto do problema:** MFEs não devem conhecer internals de cada serviço.
- **Alternativas consideradas:** Cada serviço como API pública; API Management obrigatório.
- **Decisão:** Gateway/BFF Azure Functions para roteamento e composição; chamadas internas autenticadas.

- **Justificativa, vantagens e consequências:** Ponto lógico de entrada aumenta coordenação; não pressupõe serviço pago.
- **Rastreabilidade:** RT-03; Fig. 03 e 09.

ADR 07 — Reutilização local de MongoClient

- **Status:** Aceita.
- **Contexto do problema:** Escala serverless pode multiplicar conexões e overhead de TLS.
- **Alternativas consideradas:** Instanciar/fechar driver em cada requisição.
- **Decisão:** Pool reutilizável por instância e serviço com limites de concorrência.
- **Justificativa, vantagens e consequências:** Evita overhead, mas múltiplas instâncias mantêm pools independentes; não garante limite global.
- **Rastreabilidade:** RT-14; Fig. 09.

ADR 08 — Gestão externa de segredos

- **Status:** Aceita.
- **Contexto do problema:** Conexões, JWT e IA exigem segredos sensíveis.
- **Alternativas consideradas:** Credenciais hardcoded ou embutidas no frontend.
- **Decisão:** Environment settings por serviço e proteção de repositório; chave Gemini opcional.
- **Justificativa, vantagens e consequências:** Operação de rotação e menor privilégio; depender de gestão segura de configuração.
- **Rastreabilidade:** RC-12; Fig. 09.

ADR 09 — Microsserviços e Database per Service

- **Status:** Aceita.
- **Contexto do problema:** Separar ciclo da conta, orçamento e monitoramento com autonomia de evolução.
- **Alternativas consideradas:** Um único backend e uma base compartilhada.
- **Decisão:** Identity/Budget/Monitoring como donos exclusivos de dados e APIs/eventos.
- **Justificativa, vantagens e consequências:** Mais rede e consistência eventual, porém fronteiras explícitas de responsabilidade.
- **Rastreabilidade:** RT-04; Fig. 03, 10, 12.

ADR 10 — Event-Driven Architecture com Storage Queues

- **Status:** Aceita.
- **Contexto do problema:** Operações de coleta e privacidade não devem depender de cadeia síncrona longa.
- **Alternativas consideradas:** HTTP síncrono encadeado; barramento pago com fan-out implícito.
- **Decisão:** 3 Azure Storage Queues, publicação explícita por destino, mensagens JSON V1.
- **Justificativa, vantagens e consequências:** Ordem e entrega não garantidas como exatamente uma vez; retry e roteamento por tipo.
- **Rastreabilidade:** RT-06; Fig. 07, 08.

ADR 11 — Transactional Outbox por agregado

- **Status:** Aceita.
- **Contexto do problema:** Falha entre gravação de estado e publicação poderia perder eventos.
- **Alternativas consideradas:** Publicar diretamente após salvar sem registro recuperável.
- **Decisão:** Outbox embutida na mesma escrita do agregado, dispatcher independente e consumidores idempotentes.
- **Justificativa, vantagens e consequências:** Reenvio e duplicidade possíveis; limpeza/rotação de outbox e monitoramento exigidos.
- **Rastreabilidade:** RT-07; Fig. 05, 07, 10.

ADR 12 — Consistência eventual com versionamento de snapshots

- **Status:** Aceita.
- **Contexto do problema:** O Monitoring não lê db_budgets, mas precisa das regras atuais de compra.
- **Alternativas consideradas:** Consulta direta ao banco Budget; replicação sem versões.
- **Decisão:** EventoOrcamentoV1 transporta snapshot e versão; Monitoring faz upsert somente se mais recente.
- **Justificativa, vantagens e consequências:** Janela de atraso, resultados antigos recusados e necessidade de reconciliação.
- **Rastreabilidade:** RT-17; Fig. 07, 10.

ADR 13 — Encerramento distribuído orientado a confirmações

- **Status:** Aceita.
- **Contexto do problema:** Identidade, orçamento e monitoramento armazenam dados próprios.
- **Alternativas consideradas:** Excluir tudo numa pseudo-transação global; fire-and-forget sem confirmação.
- **Decisão:** Identity bloqueia acesso, distribui por duas filas, exige ACKs distintos e marcadores por usuário.
- **Justificativa, vantagens e consequências:** Conclusão eventual, status pendente, observabilidade de retries e retenção legal.
- **Rastreabilidade:** RT-10, RC-03; Fig. 08.

ADR 14 — Clean Architecture combinada com Vertical Slice

- **Status:** Aceita.
- **Contexto do problema:** Separar infraestrutura de domínio sem perder coesão por caso de uso.
- **Alternativas consideradas:** Arquitetura em camadas com controllers/services globais acoplados.
- **Decisão:** Domínio puro, portas internas, adaptadores e slices independentes por função.
- **Justificativa, vantagens e consequências:** Mais interfaces iniciais, melhor testabilidade e autonomia de mudança.
- **Rastreabilidade:** RC-01, RC-02; Fig. 04, 12.

10. Requisitos de Qualidade

Esta seção relaciona os atributos da ISO/IEC 25010 a cenários de qualidade mensuráveis e analisa compromissos arquiteturais pelo método ATAM. Limites temporais e percentis representam **critérios de aceitação da solução**, distintos de estatísticas de operação observadas.

10.1 Árvore de Qualidade (ISO/IEC 25010)

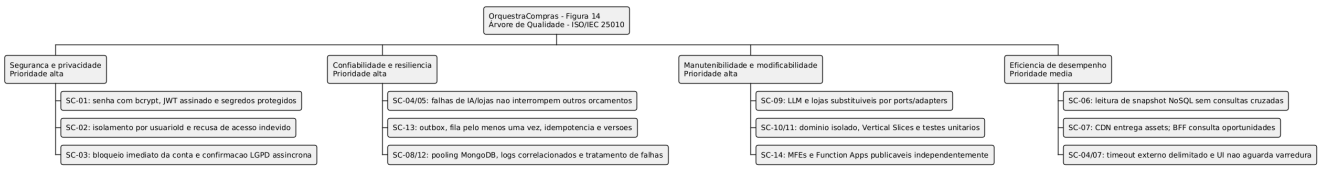


Figura 14: Árvore de Qualidade ISO/IEC 25010 — atributos prioritários.

A árvore prioriza quatro ramos: segurança e privacidade; confiabilidade e resiliência; manutenibilidade e modificabilidade; e eficiência de desempenho. Os cenários SC-13 e SC-14 abrangem especificamente a consistência dos eventos e a autonomia de implantação.

10.2 Cenários de Qualidade Mensuráveis

Tabela 31: Cenários de qualidade mensuráveis SC-01 a SC-14.

ID	Atributo	Estímulo	Ambiente	Resposta esperada	Critério de teste (meta)	Prior.
`SC-01`	Segurança	Login com credenciais válidas	Produção via HTTPS	bcrypt, JWT com claims mínimos, segredo protegido	Nenhuma senha plana em banco/log; teste de autenticação	Alta
`SC-02`	Segurança	Tentar ler/alterar orçamento alheio	Usuário autenticado	Serviço proprietário nega titularidade	100% dos testes negativos retornam negação, sem vazamento	Alta
`SC-03`	Privacidade	Titular solicita encerramento	Falhas eventuais de fila	Bloqueio imediato de novos acessos; status pendente e ACKs posteriores	Nenhum novo acesso após aceite; convergência demonstrada em teste de reconciliação	Alta
`SC-04`	Resiliência	Gemini retorna 429/timeout/JSON inválido	Ciclo periódico	Fallback local ou registro sem dados	Sem oportunidade inválida e sem falha em cascata	Alta
`SC-05`	Confiabilidade	Uma loja fora do ar	Varredura de múltiplos varejistas	Isola loja, registra ausência, avalia outras fontes	Ciclo não bloqueia outros orçamentos	Alta
`SC-06`	Eficiência	Consulta orçamento com itens e lojas	Consulta normal	Leitura de documentos embutidos	Sem consulta cruzada a outro banco e tempo aferido em teste	Média

ID	Atributo	Estímulo	Ambiente	Resposta esperada	Critério de teste (meta)	Prior.
SC-07	UX / desempenho	Usuário abre painel de oportunidades	Rede 4G simulada	Shell + MFE carregam, GET consulta Monitoring	P95 de carregamento <= 3 s nas condições definidas para o teste	Média
SC-08	Operação	Múltiplas Functions acessam MongoDB	Concorrência de produção	Pool reutilizado por instância com limites	Conexões abaixo da cota verificada em conta; alertas	Alta
SC-09	Modificabilidade	Trocar Gemini por mock/outro provedor	Mudança de integração	Trocar adaptador; domínio permanece independente	Testes de domínio não mudam por provider	Alta
SC-10	Manutenibilidade	Adicionar categoria e alternativa	Evolução de esquema	Contratos compatíveis e objetos de valor validados	Build/contratos passam sem alteração dos outros serviços	Média
SC-11	Testabilidade	Executar testes de regras	CI	Mocks de filas, loja e IA	Suite do domínio aprovada e cobertura apurada	Média
SC-12	Observabilidade	Falha em evento/Function	Operação distribuída	Logs correlationId/eventId sem PII	Falha rastreável do producer ao consumidor	Média
SC-13	Confiabilidade EDA	Entrega duplicada e fora de ordem	Fila at-least-once	Versão/idempotência impedem efeito duplicado	Uma única oportunidade lógica e snapshot mais recente	Alta
SC-14	Modificabilidade	Deploy de MFE/Function App individual	Homologação	Publica sem obrigar republicação dos demais	Pipeline e smoke tests independentes aprovados	Alta

Protocolo de medição: os ensaios registram versão de build, quantidade de usuários e orçamentos, tamanho dos payloads, região da nuvem, limites de concorrência, percentis P50/P95 e incidentes. O timeout externo de 10 segundos é uma configuração de proteção; o tempo de fallback local inferior a 50 ms é um critério de desempenho avaliado separadamente da latência da LLM. Resultados de medições são registrados por execução para preservar a rastreabilidade.

10.3 Análise de Trade-Offs Arquiteturais (Método ATAM)

ATAM-01: FaaS e Function Apps independentes

Tabela 32: Avaliação arquitetural ATAM-01.

Item de análise	Resultado
Decisão	FaaS e Function Apps independentes
Benefício principal	Escalabilidade sob demanda e manutenção separada
Custo / sensibilidade	Cold start, quota, múltiplos deployments, chamadas internas
Mitigação ou controle	Limits de concorrência, smoke tests e orçamento de custos
Rastreabilidade	RT-01 / SC-08 / SC-14

ATAM-02: LLM opcional, mock e fallback

Tabela 33: Avaliação arquitetural ATAM-02.

Item de análise	Resultado
Decisão	LLM opcional, mock e fallback
Benefício principal	Resiliência, capacidade de demonstração sem dependência externa
Custo / sensibilidade	Mock pode não representar qualidade de dados reais; heurística pode ser subótima
Mitigação ou controle	RegrasDeCompra obrigatórias; marcar origem e testes de catálogo real
Rastreabilidade	RF12–RF13 / SC-04

ATAM-03: MongoDB documental por contexto

Tabela 34: Avaliação arquitetural ATAM-03.

Item de análise	Resultado
Decisão	MongoDB documental por contexto
Benefício principal	Leitura eficiente de agregado e posse clara de dados
Custo / sensibilidade	Duplicação de snapshots, eventualidade, índices/limites do plano
Mitigação ou controle	Versão de eventos, atualização condicional e políticas de expurgo
Rastreabilidade	RT-04–05 / SC-06 / SC-13

ATAM-04: Microfrontends com Web Shell

Tabela 35: Avaliação arquitetural ATAM-04.

Item de análise	Resultado
Decisão	Microfrontends com Web Shell
Benefício principal	Deploy autônomo e separação de contas/orçamentos
Custo / sensibilidade	Peso adicional de assets, contratos, runtime de módulos remotos
Mitigação ou controle	Manifest versionado, compatibilidade e smoke test do Shell
Rastreabilidade	RT-02 / SC-07 / SC-14

ATAM-05: Filas + outbox + privacidade assíncrona

Tabela 36: Avaliação arquitetural ATAM-05.

Item de análise	Resultado
Decisão	Filas + outbox + privacidade assíncrona
Benefício principal	Tolerância a falhas e não bloquear UI
Custo / sensibilidade	Mensagens repetidas, difícil rastreamento e conclusão tardia
Mitigação ou controle	Marcadores e ACKs, idempotência, reconciliação e métricas de backlog
Rastreabilidade	RT-06–10 / SC-03 / SC-13

10.4 Análise dos Resultados ATAM

A análise ATAM evidencia compromissos entre autonomia, resiliência, consistência eventual, observabilidade e complexidade operacional. A distribuição por domínio isola responsabilidades, mas aumenta as necessidades de coordenação e diagnóstico. Os **pontos sensíveis** são (i) taxa de publicação/outbox; (ii) idade das filas e poison messages; (iii) retenção do marcador de privacidade; (iv) compatibilidade de evento/snapshot; (v) latência de validação de conta pelo BFF; e (vi) contrato de carregamento dos MFEs. Nenhum desses mecanismos pode ser validado somente por diagramas; requer testes operacionais descritos na seção 8.9.

10.5 Matriz de Priorização dos Atributos de Qualidade

Tabela 37: Prioridades dos atributos de qualidade.

Ordem	Atributo	Justificativa
1	Segurança e privacidade	Dados pessoais, credenciais e ações financeiras exigem isolamento e controle de acesso.
2	Consistência e confiabilidade	Mensagens duplicadas não podem criar oportunidades falsas nem reativar contas.
3	Resiliência e disponibilidade	Falhas de Gemini ou de uma loja não devem afetar operações básicas.
4	Modificabilidade	Microsserviços, microfrontends, Clean Architecture e Vertical Slice permitem mudanças isoladas por domínio.
5	Eficiência e custos	Pesquisa em background e hospedagem gratuita dependem de limites e medição de consumo.

11. Riscos e Débitos Técnicos

Os riscos abaixo representam ameaças e limitações associadas ao funcionamento distribuído do OrquestraCompras. Probabilidades e impactos são classificados qualitativamente, segundo os cenários de falha e as características dos serviços utilizados.

11.1 Riscos Arquiteturais e Mitigações

Tabela 38: Riscos arquiteturais e planos de mitigação.

ID	Categoria	Risco	Prob.	Impacto	Mitigação
`R-01`	LLM	Quota, 429, timeout, resposta inválida	Alta	Alto	Circuit Breaker, mock, fallback, validação RegrasDeCompra.
`R-02`	MongoDB	Pool e limite de conexões/cota do cluster	Média	Alto	Pool por instância, limitar concorrência, alarmes e plano de capacidade.
`R-03`	Scraping	Mudança HTML, anti-bot, indisponibilidade	Alta	Alto	Preferir APIs autorizadas, adaptadores e fontes simuladas rotuladas.
`R-04`	Serverless	Cold start, falhas de runtime e quota	Média	Médio	Telemetria de P95, timeout, retry e alternativas de plano.
`R-05`	Segurança	Segredos no Git ou em bundles	Média	Crítico	Varredura de secrets, Application Settings, credenciais por serviço.
`R-06`	EDA	Perda lógica, duplicação e ordem invertida	Alta	Alto	Outbox, ids, versões, idempotência, poison queues e reconciliação.
`R-07`	Privacidade	Evento tardio reativa dados encerrados	Média	Crítico	operacoesPrivacidade, bloqueio de conta e testes de atraso.
`R-08`	Contratos	MFE/BFF/serviços com versões incompatíveis	Média	Alto	Testes de contrato, manifest de módulos, rollout/rollback compatíveis.
`R-09`	Custos	Cotas e tarifas acima do esperado	Média	Alto	Alertas de consumo, limites por ambiente e plano de contingência.
`R-10`	Complexidade	Sobrecarga de contratos e coordenação entre múltiplos serviços	Média	Alto	Ownership por domínio, vertical slices, contratos mínimos, automação e rastreamento distribuído.
`R-11`	LGPD	Retenção excessiva ou anonimização incompatível com obrigação legal	Média	Alto	Inventário de dados, política por finalidade/base legal e revisão especializada.
`R-12`	Disponibilidade	Identity indisponível durante checagem de conta	Média	Alto	Timeout e falha segura, observar latência e políticas de revogação.

11.2 Débitos Técnicos Identificados e Planos de Ação

A modularidade distribuída introduz compromissos de manutenção e melhorias operacionais contínuas. Estes itens representam **débitos e pontos de atenção arquiteturais**, e não uma relação de componentes ausentes.

Tabela 39: Débitos técnicos e ações corretivas prioritárias.

ID	Débito ou ponto de atenção	Consequência arquitetural	Estratégia de controle e melhoria
`DT-01`	Versionamento de microfrontends remotos	Divergência entre manifesto do Web Shell e módulos publicados pode causar falha de carregamento.	Testes de compatibilidade, publicação versionada, cache controlado e rollback independente.
`DT-02`	Chamadas de autorização via Identity	A validação do estado da conta adiciona latência e dependência à borda.	Limites de timeout, autenticação interna, métricas e política *fail-closed*.
`DT-03`	Crescimento da outbox embutida	Eventos aguardando publicação podem aumentar o tamanho do agregado BSON.	Janela de reenvio, limpeza controlada de eventos confirmados, monitoramento de idade e tamanho.
`DT-04`	Manutenção dos adaptadores de catálogo	Mudanças nos catálogos e proteções anti-bot reduzem a estabilidade da coleta.	Preferência por APIs autorizadas, testes contratuais e fontes de contingência identificadas.
`DT-05`	Concorrência serverless e pool MongoDB	Escalonamento de Functions pode elevar conexões e latência.	Limites de instâncias/pool, indicadores P95 e alarmes por consumo de conexão.
`DT-06`	Rastreamento de eventos distribuídos	Diagnóstico atravessa produtores, filas, consumidores e bancos diferentes.	Correlação ponta a ponta com `correlationId`, `eventId` e painéis operacionais.
`DT-07`	Retenção de dados e auditoria	A retenção ilimitada eleva custos e riscos de privacidade.	Políticas por finalidade, prazo de retenção, anonimização e trilhas mínimas necessárias.
`DT-08`	Contratos compartilhados entre serviços	Mudanças de evento/API exigem coordenação entre consumidores independentes.	Schemas versionados, compatibilidade retroativa e testes de contrato por serviço.
`DT-09`	Reprodutibilidade de catálogos e IA	O comportamento de fontes externas varia e dificulta testes determinísticos.	Mock local, cenários fixos, proveniência e diferenciação explícita de resultados simulados.
`DT-10`	Consistência eventual no encerramento	Confirmações podem chegar fora de ordem ou permanecer em repetição.	Tombstones, marcadores idempotentes, reconciliação e alarmes para operações com atraso.

11.3 Matriz de Riscos Relacionados à Segurança da Informação

Tabela 40: Ameaças de segurança e mecanismos de defesa.

Superfície	Vetor	Efeito indesejado	Defesa e evidência de teste
JWT/browser	XSS ou token roubado	Acesso indevido	Token em memória, CSP, expiração, revogação e testes XSS.
API/BFF	IDOR e usuárioId falsificado	Consulta de dado alheio	Vínculo de titularidade validado no serviço dono do dado.
NoSQL	Operadores maliciosos e falta de schema	Consulta indevida	Validação de tipos, allowlist e testes de injeção.
Lojas/LLM	Prompt injection em textos externos	Bypass das regras	Tratar como dados; validar candidatos por regras de domínio.
Segredos	Commit de URI JWT/Gemini	Comprometimento de infraestrutura	Scanner de segredos, rotação e menor privilégio.
Filas	Evento falso, duplicado, atrasado	Alteração indevida/reativação	Permissões por produtor, schema, idempotência e tombstones.
Privacidade	Falha parcial do encerramento	PII residual	ACKs por serviço, reconciliação e retenção aprovada.
Infraestrutura	Conta sem limitação de custo	Cobrança ou indisponibilidade por cota	Monitoramento de consumo, alarmes e limitação de concorrência.

12. Glossário

O glossário apresenta definições dos termos, padrões e siglas utilizados na especificação, facilitando sua consulta por equipes de engenharia e partes interessadas.

Tabela 41: Glossário de termos e siglas utilizados.

Termo / sigla	Significado no contexto da solução
ADR	Registro da decisão arquitetural, alternativas e consequências.
arc42	Template estruturado para documentar arquitetura de software.
ATAM	Método de análise de compromissos entre atributos de qualidade.
BFF	Backend for Frontend; camada pública de API orientada aos MFEs.
C4 Model	Modelagem hierárquica: contexto, contêineres, componentes e código.
Clean Architecture	Organização do código com dependências apontando para o domínio e interfaces internas.
Circuit Breaker	Controle de tentativas de acesso a dependência externa instável (CLOSED/OPEN/HALF_OPEN).
Consistência eventual	Atualizações entre serviços convergem após publicação e consumo de eventos.
Database per Service	Cada microsserviço possui seus dados e credenciais exclusivas.
EDA	Arquitetura orientada a eventos, com produtores/consumidores desacoplados.
EventId / operacaoId	Identificadores de mensagem e de operação usados em correlação e idempotência.
FaaS	Function-as-a-Service: execução sob demanda acionada por triggers.
Fallback heurístico	Processamento local determinístico utilizado quando a IA não está disponível ou não é adequada.
Idempotência	Repetir uma mensagem não deve produzir efeitos adicionais indevidos.
JWT	JSON Web Token assinado e com prazo de expiração para autenticação/autorização.
LGPD	Lei Geral de Proteção de Dados Pessoais do Brasil.
LLM	Large Language Model; modelo de linguagem usado para sugerir combinações candidatas.
Microfrontend / MFE	Frontend autônomo publicado separadamente e composto pelo Web Shell.
Mock	Implementação local/simulada do contrato de integração, rotulada para desenvolvimento.
MongoDB Atlas	Serviço gerenciado de banco documental utilizado pela solução.
Outbox	Registro persistente de evento a publicar, gravado junto ao estado do agregado.
Poison Queue	Destino de mensagens que excederam tentativas de entrega/consumo.
Snapshot	Cópia versionada das regras do orçamento enviada ao Monitoring.
Tombstone / marcador de privacidade	Registro local que impede eventos tardios de reativarem dados de conta encerrada.
Vertical Slice	Agrupamento do código por caso de uso e respectivo fluxo de ponta a ponta.
Web Shell	Hospedeiro frontend que carrega MFEs e compõe navegação.
ISO/IEC 25010	Modelo de qualidade de produto de software referenciado nos atributos e cenários.